



PDF Download
3797276.pdf
24 March 2026
Total Citations: 0
Total Downloads: 268

Latest updates: <https://dl.acm.org/doi/10.1145/3797276>

RESEARCH-ARTICLE

LLM-as-a-Judge for Software Engineering: Literature Review, Vision, and the Road Ahead

JUNDA HE, Singapore Management University, Singapore City, Singapore

JIEKE SHI, Singapore Management University, Singapore City, Singapore

TERRY YUE ZHUO, Monash University, Melbourne, VIC, Australia

CHRISTOPH TREUDE, Singapore Management University, Singapore City, Singapore

JIAMOU SUN, Commonwealth Scientific and Industrial Research Organisation, Canberra, ACT, Australia

ZHENCHANG XING, The Australian National University, Canberra, ACT, Australia

[View all](#)

Open Access Support provided by:

[Singapore Management University](#)

[Monash University](#)

[Commonwealth Scientific and Industrial Research Organisation](#)

[The Australian National University](#)

Published: 19 February 2026
Accepted: 29 January 2026
Revised: 11 January 2026
Received: 30 September 2025

[Citation in BibTeX format](#)

LLM-as-a-Judge for Software Engineering: Literature Review, Vision, and the Road Ahead

JUNDA HE and JIEKE SHI, Singapore Management University, Singapore

TERRY YUE ZHUO*, Monash University & CSIRO's Data61, Australia

CHRISTOPH TREUDE, Singapore Management University, Singapore

JIAMOU SUN, CSIRO's Data61, Australia

ZHENCHANG XING, CSIRO's Data61 & Australian National University, Australia

XIAONING DU, Monash University, Australia

DAVID LO, Singapore Management University, Singapore

The rapid integration of Large Language Models (LLMs) into software engineering (SE) has revolutionized tasks from code generation to program repair, producing a massive volume of software artifacts. This surge in automated creation has exposed a critical bottleneck: the lack of scalable and reliable methods to evaluate the quality of these outputs. Human evaluation, while effective, is very costly and time-consuming. Traditional automated metrics like BLEU rely on high-quality references and struggle to capture nuanced aspects of software quality, such as readability and usefulness. In response, the LLM-as-a-Judge paradigm, which employs LLMs for automated evaluation, has emerged. This approach leverages the advanced reasoning and coding capabilities of LLMs themselves to perform automated evaluations, offering a compelling path toward achieving both the nuance of human insight and the scalability of automated systems. Nevertheless, LLM-as-a-Judge research in the SE community is still in its early stages, with many breakthroughs needed.

This forward-looking SE 2030 paper aims to steer the research community toward advancing LLM-as-a-Judge for evaluating LLM-generated software artifacts, while also sharing potential research paths to achieve this goal. We provide a literature review of existing SE studies on LLM-as-a-Judge and envision these frameworks as reliable, robust, and scalable human surrogates capable of evaluating software artifacts with consistent, multi-faceted assessments by 2030 and beyond. To validate this vision, we analyze the limitations of current studies, identify key research gaps, and outline a detailed roadmap to guide future developments of LLM-as-a-Judge in software engineering. While not intended to be a definitive guide, our work aims to foster further research and adoption of LLM-as-a-Judge frameworks within the SE community, ultimately improving the effectiveness and scalability of software artifact evaluation methods.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Software and its engineering** → **Software functional properties**; • **Computing methodologies** → **Artificial intelligence**.

Additional Key Words and Phrases: Large Language Models, Software Engineering, LLM-as-a-Judge, Research Roadmap

*Corresponding author

Authors' Contact Information: Junda He, jundahe.2022@phdcs.smu.edu.sg; Jieke Shi, jiekeshi@smu.edu.sg, Singapore Management University, Singapore; Terry Yue Zhuo, terry.zhuo@monash.edu, Monash University & CSIRO's Data61, Australia; Christoph Treude, ctreude@smu.edu.sg, Singapore Management University, Singapore; Jiamou Sun, Frank.Sun@data61.csiro.au, CSIRO's Data61, Australia; Zhenchang Xing, zhenchang.xing@data61.csiro.au, CSIRO's Data61 & Australian National University, Australia; Xiaoning Du, xiaoning.du@monash.edu, Monash University, Australia; David Lo, davidlo@smu.edu.sg, Singapore Management University, Singapore.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/2-ART

<https://doi.org/10.1145/3797276>

1 Introduction

Recent advances in Large Language Models (LLMs) [3, 13, 61, 130] are transforming the field of Software Engineering (SE) [18, 21, 27, 31, 88]. These models have demonstrated exceptional performance on longstanding challenges, from code generation [52, 131] and code summarization [2, 83] to program translation [63, 114] and repair [36, 38]. Consequently, LLMs have powered a series of new applications that streamline the software development process [4, 22].

However, the proliferation of LLMs in software development has led to a surge of generated software artifacts. Evaluating this massive output is a non-trivial task, as the inherent complexity and open-endedness of these artifacts present a profound assessment challenge. This raises a new challenging question for software engineering research:

How can we comprehensively and scalably evaluate the quality of LLM-generated software artifacts?

While human evaluation by experienced developers remains the ideal approach for assessing the quality and effectiveness of these LLM-generated software artifacts, rigorous human evaluation faces significant practical hurdles. A survey of SE researchers revealed that 84% agree that human evaluation is problematic, primarily due to time constraints, the need for specialized practical knowledge, and the high cost involved [8]. Additionally, human evaluators may experience fatigue and reduced concentration, which further undermines the quality of the evaluation and delays the evaluation process [42].

To overcome these challenges, researchers often turn to automated metrics [30, 50, 64], which offer scalability at a lower cost. Yet, traditional automated metrics have their own drawbacks. For example, Pass@ k is a widely used metric for measuring code correctness in code generation, which involves executing the first k generated code snippets against unit tests. Although useful, Pass@ k demands substantial human effort to design comprehensive test suites and configure execution environments. For other tasks like code summarization and translation, researchers often use text-similarity metrics (e.g., BLEU [64], CodeBLEU [73]) and embedding-based metrics (e.g., BERTScore [118], CodeBERTScore [123]). However, the effectiveness of these metrics also relies heavily on high-quality references that are typically annotated by human experts.

More broadly, traditional automated metrics are ill-equipped to handle the inherent complexity of LLM-generated content. They struggle to assess the nuanced aspects, such as naturalness, usefulness, and adherence to best practices. This fundamental limitation causes their results to frequently misalign with human judgment, a finding confirmed by several studies [17, 30, 42, 74]. Consequently, achieving comprehensive evaluation while ensuring scalable automation remains a persistent challenge in assessing the quality of LLM-generated software artifacts.

Recently, the remarkable success of LLMs has inspired the emergence of the LLM-as-a-Judge (LLM-J) paradigm [121]. This paradigm extends LLMs' role from content generation to content evaluation, positioning them as scalable and cost-effective surrogates for human experts. Several key attributes make LLMs particularly well-suited for this evaluation role. First, numerous studies have shown that LLMs exhibit both impressive coding abilities [45, 51, 56] and human-like reasoning skills [25, 32, 66]. Additionally, LLMs are often trained through reinforcement learning from human feedback (RLHF) [5], which helps in closely aligning LLM's judgments with human expert judgments [10]. Moreover, unlike human evaluators, LLMs do not experience fatigue or reduced efficiency from prolonged work, allowing them to maintain consistent performance over extended periods. Together, these attributes collectively make LLMs compelling candidates for automated evaluation.

The SE community is increasingly exploring LLM-as-a-Judge to overcome the limitations of both costly human evaluation and traditional automated metrics. A growing body of research work demonstrates the promise of this approach [1, 89, 99, 129]. Nevertheless, the research area remains in its early stages, and significant breakthroughs

are required to realize the full potential of this paradigm by 2030 and beyond. Therefore, this paper aims to present a forward-looking vision and a concrete research roadmap that inspires further progress.

In the following sections, this paper first provides a formal definition of the LLM-as-a-Judge concept (Section 2). We then review key historical milestones and recent advances in LLM-as-a-Judge for software engineering (Section 4). Looking beyond 2030, we envision LLM-as-a-Judge systems as reliable, robust, and scalable human surrogates capable of evaluating a wide range of software artifacts while providing consistent, multi-faceted assessments. To validate this vision, we identify key limitations and research gaps, outlining a roadmap with specific research directions and potential solutions for the SE research community to pursue (Section 5). In summary, our study makes the following key contributions:

- A systematic review of 42 primary studies on the use of LLM-as-a-Judge in software engineering, providing a comprehensive summary of its applications.
- An analysis of the limitations of existing research, identifying key challenges and research gaps in the field.
- A forward-looking vision for LLM-as-a-Judge in SE toward 2030 and beyond, supported by a detailed research roadmap to guide future exploration.

2 Definition of LLM-as-a-Judge

As a recent emerging topic, various definitions of LLM-as-a-Judge are currently proposed and not yet unified. This paper was inspired by the definition proposed by Li et al. [44]. Informally speaking, LLM-as-a-Judge utilizes LLMs as evaluative tools to assess software artifact based on predefined evaluation criteria. Formally speaking, LLM-as-a-Judge is defined as follows:

$$E(\mathcal{T}, \mathcal{C}, \mathcal{X}, \mathcal{R}) \rightarrow (\mathcal{Y}, \mathcal{E}, \mathcal{F}) \quad (1)$$

The function E is the core evaluation mechanism powered by LLMs. It takes the following key **inputs**:

- $\mathcal{T}$ – Evaluation Type. It mainly covers three types: *point-wise*, *pairwise*, and *list-wise*.
 - **Point-wise Evaluation:** Function E evaluates a single candidate content independently and often provides a score or categorical classification.
 - **Pair-wise Evaluation:** Function E compares two candidates and determines their relative quality.
 - **List-wise Evaluation:** Function E ranks multiple (more than 2) candidates.
- $\mathcal{C}$ – Evaluation Criteria. It describes the assessment aspects, e.g., correctness, helpfulness, readability, etc.
- $\mathcal{X}$ – Evaluation Item. It is the content to be judged. Note that $\mathcal{X}$ can be a single candidate or a set of candidates $(x_1, x_2, \dots, x_n)$, where x_i is the i^{th} candidate to be evaluated.
- $\mathcal{R}$ – Reference (optional). For instance, for a code generation task, $\mathcal{R}$ can be a code implementation that successfully fulfills the requirements of a coding task. Note that $\mathcal{R}$ is not mandatory, many LLM-as-a-Judge systems operate in a reference-free manner.

Given these inputs, E can produce three primary **outputs**:

- $\mathcal{Y}$ – Evaluation Result. $\mathcal{Y}$ can take one of the following forms:
 - **Graded Evaluation:** Each candidate is assigned a score, either numerical (discrete or continuous) or categorical.
 - **Rank Ordering:** Candidates $(x_1, x_2, \dots, x_n)$ are ranked based on their assessed quality.
 - **Best-Choice Selection:** Function E selects the most suitable candidate (x_i) from the given set $(x_1, x_2, \dots, x_n)$.
- $\mathcal{E}$ – Evaluation Explanation. Optionally provides reasoning and justification for the evaluation.
- $\mathcal{F}$ – Feedback. Optionally offers constructive suggestions for improving the evaluated item $\mathcal{X}$.

In general, we envision LLM-as-a-Judge as a genuine surrogate for human evaluation. It should support a wide spectrum of nuanced assessments (e.g., usefulness, adequacy, and conciseness) and ideally provide justifications or constructive feedback.

Selecting the appropriate evaluation paradigm depends fundamentally on the nature of the SE task and the availability of comparative candidates. For example, Point-wise evaluation is an absolute assessment paradigm. It is the only viable option for tasks such as Binary Vulnerability Classification, where the LLM judge must determine if a specific code snippet is 'secure' or 'vulnerable' based solely on its intrinsic properties, without reference to external candidates. Beyond task constraints, the choice involves balancing precision against efficiency. Point-wise evaluation is ideal for absolute assessments, such as verifying whether a specific solution meets requirements. However, it often suffers from calibration inconsistency, where LLMs struggle to assign distinct numerical scores to similar artifacts.

To mitigate calibration issues, relative comparison paradigms (Pair-wise & List-wise evaluation) simplify the task to ranking candidates, yielding higher alignment with human preference. For example, Pair-wise Evaluation is widely considered the more robust method for minimizing reasoning errors, as the LLM focuses on only two inputs at a time. However, it is also sensitive to presentation effects (order/position bias). List-wise evaluation offers a distinct efficiency advantage when ranking multiple candidates. The primary limitation is the "lost-in-the-middle" phenomenon: as list length increases, the model's attention degrades, often causing valid candidates positioned in the middle of the input to be overlooked.

3 Review Methodology

This section presents the methodology of our systematic literature review. We first describe the survey scope along with the inclusion and exclusion criteria, then introduce the data sources, search strategy, and snowballing procedure used to achieve comprehensive coverage. Finally, we outline the quality assessment criteria applied to the selected studies.

3.1 Survey Scope

The scope of this review is defined by two primary dimensions: the research must focus on software engineering and must employ the LLM-as-a-Judge paradigm.

Inclusion/Exclusion Criteria. To ensure the selection of relevant and high-quality studies, we established the following inclusion and exclusion criteria:

- ✓ *The paper must be written in English.*
- ✓ *The full text of the paper must be accessible, either through open-access channels or the researchers' institutional subscriptions.*
- ✓ *The paper must utilize the LLM-as-a-Judge paradigm to one or more software engineering tasks.*
- ✗ *The paper has fewer than 5 pages.*
- ✗ *The paper is a duplicate or a minor extension of another study by the same authors.*
- ✗ *Books, keynote records, panel summaries, technical reports, theses, tool demo papers, editorials.*
- ✗ *The paper is a literature review or survey.*
- ✗ *The paper does not involve software engineering-related tasks.*
- ✗ *The paper lacks experimental results and mentions the LLM-as-a-Judge approach only in future work or discussions.*

3.2 Paper Source

For our literature search, we selected the DBLP computer science bibliography¹ as the primary data source. This choice follows established practices in systematic literature reviews (SLR) for software engineering [27, 54]. Studies [116] have shown that DBLP’s corpus broadly subsumes other major academic databases, such as IEEE Xplore² and the ACM Digital Library.³ A key advantage of DBLP is its indexing of arXiv,⁴ which allows the inclusion of recent preprints that may face delays before appearing in traditional venues. Consequently, using DBLP ensures a more comprehensive and timely collection of relevant literature for this study.

3.3 Search String Formulation

We developed the search string through a systematic and multi-stage process. First, the first three authors drew on their expertise in LLM-as-a-Judge and software engineering to curate a set of seed papers that apply LLM-as-a-Judge to software engineering tasks. We then analyzed the titles, abstracts, and keywords of these papers to extract an initial list of search terms. These terms were subsequently refined and expanded through brainstorming sessions to incorporate relevant synonyms and variations.

The final search string combines two core concepts: (1) terms related to the LLM-as-a-Judge paradigm and (2) terms related to the software engineering domain. The final query is provided in the online appendix [26]. Based on the selected keywords, we conduct 42 searches on DBLP on August 7th, 2025.

3.4 Snowballing

To ensure comprehensive coverage beyond the initial keyword search, we applied backward and forward snowballing [33, 97]. The process was conducted iteratively from August 7 to August 12, 2025, until saturation was reached.

3.5 Quality Assessment

To mitigate bias and ensure reliability, we filtered studies based on three Quality Assessment Criteria (QAC): **(QAC1) Research Contribution**, **(QAC2) Methodological Clarity**, and **(QAC3) Experimental Rigor**. These criteria assess the novelty, reproducibility, and scientific soundness of each study. The checklist was independently applied by the first three authors, with discrepancies resolved through consensus. Detailed definitions for these criteria are provided in our Online Appendix [26].

3.6 Results

Our paper selection process yielded a total of 42 relevant publications. Figure 1 illustrates their temporal distribution over 2023, 2024, and 2025. Following the first work published in 2023, the field expanded with 15 publications in 2024. This trend intensified remarkably in 2025, with 26 publications cataloged by August alone. This exponential growth strongly indicates a surging research interest in the LLM-as-a-Judge paradigm within Software Engineering. The fact that the publication count for 2025 has already far surpassed the previous year’s total in just eight months underscores the exceptional pace of development in this domain. For published works, the formal acceptance year was used as the publication date, which may differ from their initial posting on preprint servers like arXiv.

¹<https://dblp.org/>

²<https://ieeexplore.ieee.org/>

³<https://dl.acm.org/>

⁴<https://arxiv.org/>

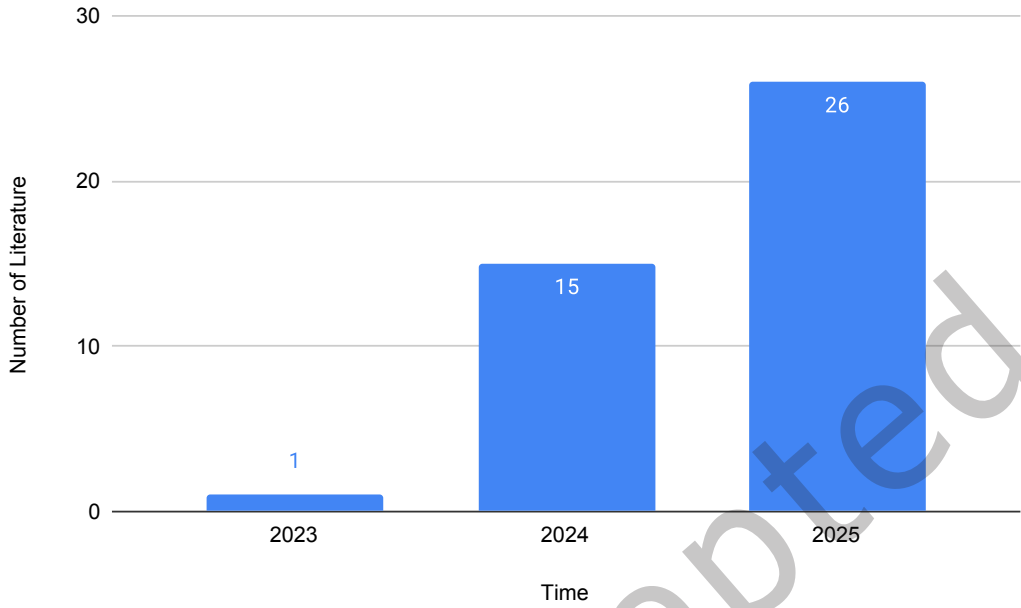


Fig. 1. Annual Distribution of Publications on LLM-as-a-Judge in SE (2023 - Aug 2025)

4 Literature Review

In this section, we systematically review the application of the LLM-as-a-Judge paradigm across a spectrum of software engineering tasks. To provide a structured overview, we categorize the existing literature based on the SE domain being addressed, as shown in Table 1. Our analysis covers four areas: **requirements engineering**, **coding assistance**, **software maintenance**, and **quality assurance**. These categories were chosen because they organically emerged from the reviewed literature and correspond to the fundamental stages of the software development life-cycle. This review enables us to pinpoint the distinct challenges characterizing the current nascent landscape of LLM-as-a-Judge in SE. These insights establish the empirical groundwork for the detailed analysis of limitations and future research opportunities that follows.

4.1 Requirements Engineering

The LLM-as-a-Judge paradigm is being effectively applied to the foundational phase of software development: requirements engineering. Approaches use LLMs to automate the evaluation of early-stage artifacts, ensuring the quality of documents before development begins. From Table 2, we can see this involves documents ranging from user stories to technical system specifications.

4.1.1 Requirements Documents. The LLM-as-a-Judge paradigm is used to assess the quality of requirement artifacts. For instance, Lubos et al. [57] use Llama 2 (70B) to evaluate software requirements against the ISO 29148 standard⁵ for nine quality characteristics: Appropriate, Complete, Conforming, Correct, Feasible, Necessary, Singular, Unambiguous, and Verifiable. They find that the LLM not only identifies most quality flaws but also

⁵<https://www.iso.org/standard/72089.html>

Table 1. A Taxonomy of LLM-as-a-Judge Literature in Software Engineering. The number of LLM-as-a-Judge publications for evaluating each kind of SE artifacts is provided in brackets. A single paper may be cited for multiple SE artifacts.

SE Activity	SE Artifact	Reference
Requirement Engineering	Requirements Documents (3)	[1, 57, 71]
	System Specification (1)	[72]
	User Stories (1)	[71]
Coding Assistance	Generated Code (27)	[39, 40, 46, 80, 86, 92, 106, 127, 128] [1, 34, 59, 65, 84, 85, 87, 89, 90, 125] [11, 20, 35, 62, 93, 102, 120, 129]
	Code Summary (8)	[1, 11, 19, 46, 60, 89, 99, 125]
	Translated Code (2)	[46, 89]
	Answers to Software Questions (2)	[101, 121]
	Bug Report (1)	[42]
Quality Assurance	Tests (3)	[35, 81, 91]
	Vulnerability Explanation (1)	[94]
Maintenance	Commit Message (1)	[115]
	Code Patches (5)	[1, 35, 48, 103, 125]

Table 2. Overview on the application of LLM-as-a-Judge in Requirements Engineering

	Evaluation Type	Evaluation Criteria	Evaluation Item	Evaluation Result
Ahmed et al. [1]	Point-wise	Presence of a causal relationship	Requirements documents	Graded
Quattrocchi et al. [71]	Point-wise	Feature Specificity, Rationale Clarity, Problem-Oriented, Language Clarity, Internal Consistency	User Stories	Graded
Lubos et al. [57]	Point-wise	ISO-29148 quality characteristics (Appropriate, Complete, Conforming, Correct, Feasible, Necessary, Singular, Unambiguous, Verifiable)	Requirements documents	Graded
Reinbold et al. [72]	Point-wise	Fulfillment and Applicability of a requirement with respect to the specification	System specification	Graded

provides reliable explanations for them. Ahmed et al. [1] demonstrate a more granular application of the LLM-as-a-Judge paradigm by using an LLM to extract causal relationships from natural language requirements. The LLM provides a binary judgment on whether a sentence contains a causal link (e.g., 'If event 1, then event 2').

4.1.2 System Specifications. Reinbold et al. [72] apply LLMs to verify technical system specifications. Their work demonstrates that models like GPT-4o and Claude 3.5 Sonnet can effectively assess whether a system specification fulfills its corresponding requirements.

4.1.3 User Stories. Quattrocchi et al. [71] employ ten LLMs to assess the semantic quality of user stories against five criteria: Feature Specificity (measuring how precisely a software feature is described), Rationale Clarity (the clarity of the feature's justification), Problem-Oriented (evaluating whether the user story specifies a problem

without implying a solution), Language Clarity (the clarity and precision of the language), and Internal Consistency (checking for contradictions).

4.2 Coding Assistant

This sub-section details its application across four primary coding assistance tasks: code generation, code summarization, code translation, and software question answering. Correspondingly, we focus on the evaluation of generated code, code summary, translated code, and answers to software questions. Table 3 provides an overview of the application of LLM-as-a-Judge in Coding Assistance.

4.2.1 Generated Code. Effectively and automatically evaluating the quality of generated code is one of the most critical problems in software engineering. The evaluation of generated code is a primary application for LLM-as-a-Judge. Before the introduction of LLM-as-a-Judge, automated code evaluation primarily relied on execution-based method, i.e., executing the code based on pre-defined test cases [10, 29, 131]. However, passing all the test cases does not necessarily mean the quality of the code is good, as the test cases may not comprehensively cover all the edge cases [15]. Constructing test cases can also be particularly challenging for certain tasks [85], such as AI model training, web scraping, etc. When test cases are unavailable, many studies rely on reference-based metrics [14, 122], such as CodeBLEU [73] and CodeBERTScore [123]. Nevertheless, the quality of the reference code restricts the effectiveness of reference-based metrics. Prior research [17] highlighted that reference-based metrics frequently misalign human judgments in code generation.

Consequently, a substantial body of research has explored the effectiveness of LLMs in evaluating code generation [65, 84, 85, 89, 106, 120, 121, 127, 129]. We highlight three key advantages of LLM-as-a-Judge compared to traditional automated metrics (such as BLEU or Pass@k):

- *Execution-free*: It does not require compulsory execution of the code and test cases, unlike metrics such as Pass@k, for example.
- *Reference-free*: Unlike reference-based metrics, LLM-as-a-Judge does not require a reference code. Even though we can still optionally provide the reference code.
- *Multi-Facet Evaluation*: LLM-as-a-Judge assesses intrinsic code qualities that traditionally required human judgment, such as readability and usefulness. Further, it can also provide qualitative, actionable feedback that explains the rationale behind its assessment.

Common Evaluation Criteria. We summarize the common evaluation criteria used in the literature for code generation. To align with standard software quality models (e.g., ISO/IEC 25010), we categorize them into two main dimensions: *Functional Properties* and *Non-Functional Properties*.

Functional Properties: This dimension assesses whether the code fulfills the specified functionalities and provides the required results. It focuses on the correctness of the code's output and behavior. Key criteria include:

- **Functional Correctness**: Verifies whether the code produces the expected output according to the task description.
- **Functional Completeness**: Assesses whether the implementation covers all aspects of the requirement, ensuring no necessary features or logic are missing.

Non-Functional Properties: These criteria evaluate attributes beyond the key functionalities, such as readability and adherence to best practices. Key criteria include:

- **Complexity & Efficiency**: Analyzes computational complexity, execution time, and memory usage.
- **Helpfulness**: Evaluates whether the code contributes meaningfully to solving the problem. This includes distinguishing partially correct solutions from completely useless and irrelevant code.
- **Readability**: Measures how easily the code can be understood, including aspects like fluency, clarity, and conciseness.

Table 3. Overview on the application of LLM-as-a-Judge in Coding Assistance

Reference	Evaluation Type	Evaluation Criteria	Evaluation Items	Evaluation Result
Agent-as-a-Judge [128]	Point-wise	Requirement Satisfaction Rate	Generated Code	Graded
Ahmed et al. [1]	Point-wise	Accuracy, Adequacy, Conciseness, Similarity	Generated Code, Code summary	Graded
AIME [65]	Point-wise	Logic, Syntax, Readability, Efficiency, Redundancy	Generated Code	Graded
Findeis et al. [20]	Pair-wise	Correctness	Generated Code	Best-Choice
CODE-DITING [106]	Point-wise	Correctness	Generated Code	Graded
CodeJudge [85]	Point-wise	Correctness	Generated Code	Graded
CodeJudge-Eval [120]	Point-wise	Correctness	Generated Code	Graded
CodeJudgeBench [35]	Point-wise, Pair-wise	Correctness	Generated Code	Best-Choice, Graded
Codeultrafeedback [93]	Point-wise, List-wise	Coding preferences, Correctness, Usefulness	Generated Code	Graded, Ranked-Ordering
CodeVisionary [90]	Point-wise	Code Correctness, Functionality, Clarity	Generated Code	Graded
Jaoua et al. [34]	Point-wise, List-wise	Relevance, Accuracy, Coverage	Generated Code	Graded, Rank Ordering
Crupi et al. [11]	Point-wise	Correctness, Adequacy, Conciseness, Fluency, Understandability	Generated Code, Code Summary	Graded
Moon et al. [59]	Point-wise	Correctness	Generated Code	Graded
JETTS [127]	Point-wise, List-wise	Correctness, Overall Quality	Generated Code	Graded, Best-Choice, Rank Ordering
Koutchme et al. [40]	Point-wise	Accuracy, Selectivity, Clarity	Generated Code	Graded
Simões et al. [80]	Point-wise	Readability, Overall Quality	Java class files	Graded
Farchi et al. [19]	Pair-wise	Usefulness, Similarity, Consistency	Code summaries	Graded
Vu et al. [87]	Pair-wise, List-wise	Correctness, Quality	Generated Code	Best-Choice
Kim et al. [39]	List-wise	Correctness	Generated Code	Best-Choice
HuCoSC [102]	Pair-wise	Logical correctness, Execution correctness, Efficiency	Generated Code	Graded
ICE-Score [129]	Point-wise	Usefulness, Correctness	Generated Code	Graded
Judgebench [84]	Pair-wise	Factual Correctness, Logical Correctness	Generated Code	Best-Choice
Li et al. [46]	Point-wise, Pair-wise	Helpfulness, Functional correctness, syntactic validity, semantic accuracy	Generated Code, Code Summary, Translated Code	Graded, Best-Choice
Vo et al. [86]	Point-wise, Pair-wise	Functionality matching, Logic correctness	Generated Code	Graded
MCTS-Judge [92]	Point-wise	Code logic, Completeness, Correctness	Generated Code	Graded, Best-Choice
Mu et al. [60]	Pair-wise	Natural Semantics, Readability, Naturalness	Generated Code, Code Summaries	Graded
Wang et al. [89]	Point-wise, Pair-wise	Functional correctness, Consistency, Readability, Idiomatic Usage	Generated Code, Code summary, Translated Code	Graded, Best-Choice
Pan et al. [62]	Point-wise	Correctness, Adherence to specification	Generated Code	Graded
CODERPE [99]	Point-wise	Coherence, Consistency, Fluency, Relevance	Code summary	Graded
Xie et al. [101]	Point-wise	Factual Content, Semantic Similarity, Fact Match Ratio	Responses to queries about Databricks documentation	Graded
Zheng et al. [121]	Point-wise, Pair-wise	Helpfulness, Relevance, Accuracy, Depth, Creativity, Details	Answers to coding questions	Graded, Best-Choice
Zhou et al. [125]	Point-wise, Pair-wise	Correctness, Semantic Equivalence	Generated Code, Code summary	Graded

- **Stylistic Consistency:** Ensures adherence to established coding standards, formatting guidelines, and best practices.
- **Reference Similarity:** Assesses how closely the generated code aligns with a given reference implementation.
- **Fault Tolerance & Error Handling:** Evaluates how the code manages edge cases, exceptions, invalid inputs, and unexpected failures.
- **Minor Anomalies & Warnings:** Identifies non-critical issues such as redundant code or unused variables that do not affect execution.

While promising, recent studies highlight significant challenges for LLM-as-a-Judge in evaluating code generation. An extensive empirical study by Crupi et al. [11] found that even powerful models like GPT-4-turbo frequently misjudge code correctness. This limitation has driven research beyond simple prompting techniques [11, 93, 129] toward more sophisticated frameworks.

One set of methods focus on utilizing prompt engineering to instruct LLM-as-a-Judge to perform an evaluation. The common methods are summarized below.

- *Structured Prompting.* This is the foundational method where the LLM is given a detailed, structured prompt that clearly defines the task, the evaluation criteria, the scoring scale, and the required output format. ICE-Score [129] is a prime example of this approach, instructing the LLM to follow specific steps to evaluate code, which proved more reliable than simple and direct prompts. CodeJudge [85] leverages a detailed taxonomy of common programming errors in the prompt to guide the LLM in analyzing the generated code.
- *Decomposition and Multi-Step Reasoning.* To move beyond superficial analysis, researchers have developed methods that prompt the LLM to engage in a more deliberate, multi-step reasoning process [86, 92, 102]. One key strategy involves simplifying the code itself. This is often achieved by first abstracting the code into a higher-level representation before judgment. For example, HuCoSC [102] mimics human cognition by performing Recursive Semantic Comprehension. It recursively decomposes code via its Abstract Syntax Tree (AST) into smaller parts. Vo et al. [86] first translate the code into pseudocode. This allows the LLM judge to assess the core algorithm without being confused by complex, language-specific syntax. Further, another approach is to decompose the evaluation process. MCTS-Judge [92] reframes evaluation as a search problem, where the goal is to find the most reliable reasoning path. It uses Monte Carlo Tree Search [7] to explore a tree of possible reasoning trajectories, where each path represents a unique sequence of evaluation sub-tasks (e.g., analyzing logic, then checking functionality).

Another set of frameworks [20, 90, 128] gives the LLM judge more autonomy (**Agentic**) and access to external information and tools. The Agent-as-a-Judge framework [128] utilizes an agentic system with modules for file system interaction, dependency graphing, and code execution. This design allows the agent to interact with the output of another agent and provide detailed, step-by-step feedback. Similarly, CodeVisionary [90] employs an agent with a suite of tools, including dynamic code execution, static linting, and unit testing. Findeis et al. [20] evaluate LLM's ability to distinguish between correct and incorrect Python solutions. They demonstrate that augmenting the LLM judge with a code execution tool increased its agreement with ground truth from below 42% to approximately 72%.

To enhance reliability and mitigate the biases of a single LLM, methods are proposed to leverage the power of multiple evaluators and then employ various strategies to combine judgments [65, 90, 125]. For instance, the SWE-Judge framework [125] aggregates evaluations scores from a dynamically assembled team of judges. Patel et al. propose AIME framework [65], which uses a specialist approach, assigning different LLMs to evaluate distinct criteria such as correctness and readability. They then theoretically prove the benefit of multiple evaluators over a single LLM evaluator. Other systems, like CodeVisionary [90], facilitate direct negotiation among multiple LLM judges to reach a final consensus on the evaluation score.

Another research direction focuses on creating smaller, more efficient, and specialized judges [87, 106]. These approaches typically aim to improve performance, reduce computational costs, and increase explainability. Key strategies include:

- *Knowledge Distillation*: This method transfers capabilities from a large model to a smaller one. For instance, the CODE-DITING framework [106] uses a reasoning-focused data distillation process to create powerful 1.5B and 7B parameter judges from a 671B reasoning model (DeepSeek-R1). The resulting compact models are highly efficient, with the 7B version surpassing the performance of much larger models like GPT-4o.
- *Fine-Tuning on Human Judgments*: This method uses a large dataset of human judgments to fine-tune the LLM Judge model. For instance, Vu et al. [87] curated the FLAME collection, a dataset with over 5.3 million human judgments across 102 diverse quality assessment tasks, which includes specialized datasets for coding that teach the model to distinguish between correct and buggy programs based on coding problems or GitHub issues. They then fine-tuned a PaLM-2-24B model on this collection to create the FLAME autorater models.

Improving Code Generation via LLM-as-a-Judge. LLM-as-a-Judge can be actively used to improve the output of code generation models. The most common applications are 1. Selection from Multiple Solutions (Verification & Reranking) and 2. Iterative Refinement with Feedback.

- *Selection from Multiple Solutions*. Kim et al. [39] used an LLM judge as a verifier to select the best solution from multiple candidates, as code generation models often exhibit a large gap between single-shot success (Pass@1) and multi-shot success (Pass@5). Kim et al. utilizes LLM-as-a-Judge to close this gap without requiring a ground-truth test suite. Vu et al. [87] trained the FLAME autorater models on a dataset of over 5.3 million human judgments across 102 quality assessment tasks, including specialized coding datasets that teach the model to distinguish correct from buggy programs using coding problems or GitHub issues. They used FLAME to improve code generation on the HumanEval Python benchmark. In this experiment, FLAME re-ranked 10 code samples generated by other models. Selecting the top-ranked sample significantly improved Pass@1. For example, it raised CodeGen-16B’s accuracy from 21.2% to 31.1%.
- *Iterative Refinement with Feedback*. Another directions uses the judge’s output as feedback to iteratively refine the code generation model. Research shows that feedback quality is critical. Zhou et al. [127] found that generic, unstructured natural-language critiques from LLM judges were largely ineffective for enabling effective code refinement. Vo et al. [86] built a Reflection code agent that used feedback to automatically fix incorrect Bash scripts. Weyssow et al. [93] used the feedback from the LLM judge to align CodeLlama-7B with coding preferences. Experiments showed improved adherence to coding style and higher functional correctness.

4.2.2 Code Summary. The LLM-as-a-Judge paradigm is particularly well-suited for evaluating code summaries. Unlike code generation, where functional correctness can be verified through execution, the quality of a summary is subjective and relies on human perception. Consequently, LLM judges show strong potential in this domain, with multiple studies confirming that their evaluations align more closely with human judgment than traditional metrics [1, 11, 99, 125].

For instance, Crupi et al. [11] found that GPT-4 achieves moderate to high agreement with human evaluators, especially when assessing content adequacy. To further enhance evaluation quality, innovative prompting strategies have been developed. CODERPE [99], for example, instructs the LLM judge to adopt different professional personas—such as a code reviewer, author, or system analyst—to assess summaries from multiple perspectives.

Common criteria for evaluating code summaries include:

- **Language-related:** Ensure clear, well-structured sentences with precise and appropriate wording.
- **Content-related:** Summarize the core functionality and logic concisely, providing sufficient detail without including unnecessary information.

Table 4. Overview on the application of LLM-as-a-Judge in Quality Assurance

	Evaluation Type	Evaluation Criteria	Evaluation Item	Evaluation Result
RUM [91]	Point-wise	Standardization, Sufficiency, Consistency, Readability, Runnability	Test case document, Test script code, Test step screenshots	Graded
LLMs as Evaluators [42]	List-wise	Factual Correctness, Completeness, No Hallucinations, Adequate Coverage	Bug report titles, Bug report summaries	Graded, Best-Choice
LLM4VV [81]	Point-wise	Syntax, Directive Appropriateness, Clause Correctness, Memory Management, Compliance, Logic	Compiler test code	Graded
R2VUL [94]	Point-wise	Completeness, Clarity, Actionability	Vulnerability Explanations	Graded

- **Effectiveness-related:** Evaluate whether the summary is useful for developers and enhances their understanding of the code.

4.2.3 Translated Code. Code translation involves converting code from one programming language to another while preserving its functionality [104]. Empirical evidence [89] suggests that the LLM-as-a-Judge paradigm is particularly effective for this evaluation task. Wang et al. [89] focused on translations involving, Java, Python, C, and C++. They found that LLM-as-a-Judge reached a Pearson correlation of 0.81 with human scores on the CodeTransOcean [104] dataset, whereas the best traditional metric ChrF++ [69], only achieved 0.34. The success of LLM-as-a-Judge is mainly attributed to the ability of LLMs to assess functional and semantic equivalence. Since traditional metrics rely on lexical matching, they often penalize correct translations that are syntactically different from a reference solution. In contrast, LLMs can understand the underlying program logic, enabling a more robust and reference-free evaluation.

4.2.4 Answers to Software Question. Zheng et al. [121] investigated using LLMs like GPT-4 to judge responses to complex, open-ended queries, including coding and reasoning tasks. They found that GPT-4’s judgments align remarkably well with human preferences, achieving over 80% agreement, which is comparable to inter-human agreement. Xie et al. [101] applied LLM-as-a-Judge to evaluate answers to questions about Databricks documentation. Their work addresses a key limitation where LLM judges often overemphasize minor details while undervaluing critical information. They introduced a prompting mechanism that instructs the judge to differentiate between “Critical Facts,” “Supporting Facts,” and “Trivial Facts.” This weighting system allows the judge to prioritize essential information and avoid penalizing answers for minor omissions, improving the human alignment rate by an average of 6%.

4.3 Quality Assurance

In software engineering, Quality Assurance (QA) encompasses a broad range of activities aimed at ensuring software quality, from identifying bugs to validating test coverage and assessing security. The LLM-as-a-Judge paradigm is being applied to automate the evaluation of several key QA artifacts, as summarized in Table 4.

4.3.1 Bug Report. For evaluating bug reports, LLM judges have shown significant promise. A study by Kumar et al. [42] found that advanced models like GPT-4o consistently matched or exceeded human accuracy in selecting the most accurate titles and summaries for bug reports. Crucially, the study noted that while human evaluators experienced fatigue that degraded their accuracy over time, the LLM’s performance remained stable.

This consistency makes LLM-as-a-Judge a scalable and potentially more reliable alternative for large-scale QA evaluations.

4.3.2 Tests. Sollenberger et al. [81] apply LLM-as-a-Judge to evaluate the compiler test suite. These are tests designed to validate and verify compiler implementations for parallel programming models of OpenMP⁶ and OpenACC⁷. They demonstrate an agent-based approach where the LLM judge is provided with compiler outputs and execution results to assess the correctness of compiler test suites. Wang et al. [91] propose RUM, a hybrid system that delegates simple, objective checks (e.g., syntax, naming conventions) to fast, rule-based systems, reserving the LLM for more subjective assessments like test case readability and coverage. Their evaluation covers a comprehensive set of testing work products, including Test Case Documents, Test Script Code, and Test Reports. This division optimizes for both cost and quality, achieving a high correlation with human expert scores.

4.3.3 Vulnerability Explanation. Instead of making direct and high-stakes vulnerability judgments, as explored in prior studies [124, 126], LLM-as-a-Judge are increasingly employed in meta-evaluation roles. For example, recent work [94] demonstrates that they can assess whether vulnerability explanations are logically consistent, sufficiently detailed, and clearly articulated. In this way, LLMs serve less as ultimate arbiters of security and more as scalable reviewers that enhance the reliability and interpretability of automated vulnerability analysis.

4.4 Maintenance

Software maintenance, which involves modifying and updating code after its initial release, is another critical area where the LLM-as-a-Judge paradigm is applied. The focus here is on evaluating the quality of the artifacts that document and implement these changes, namely commit messages and code patches. Table 5 provides an overview of these studies.

4.4.1 Commit Message. The LLM-as-a-Judge paradigm has proven to be a effective method for evaluating the quality of automatically generated commit messages [115]. Zeng et al. [115] demonstrated that when using advanced prompting strategies combining Chain-of-Thought and few-shot examples, LLMs like GPT-4 can achieve evaluation performance comparable to human experts. The study assessed messages on their ability to explain both “what” a change does and “why” it was made. The LLM-based approach significantly outperformed traditional reference-based metrics (e.g., BLEU, ROUGE), effectively overcoming the problem where multiple, lexically different commit messages can be equally valid for a single code change. Crucially, the research also confirmed the stability of the LLM evaluator, showing acceptable levels of reproducibility and robustness.

4.4.2 Code Patch. The LLM-as-a-Judge paradigm is increasingly applied to the evaluation of code patches [1, 48, 103, 125], a critical task in automated program repair and vulnerability mitigation. Yadavally et al. [103] achieve high accuracy by using an LLM to predict, in an execution-free manner, whether a given patch would pass unseen tests, while Ahmed et al. [1] employ LLMs to judge code patches aimed at fixing static analysis warnings, a task requiring deep reasoning. More recently, Li et al. [48] introduce CleanVul, which leverages LLM heuristics to identify security-related patches and construct a high-quality dataset of them, showing that LLM-based judgments can significantly improve the correctness and generalizability of downstream vulnerability detectors. More broadly, Zhou et al. [125] propose SWE-Judge, an ensemble-based LLM-as-a-Judge metric that integrates multiple evaluation strategies (e.g., direct assessment, semantic equivalence checking, and test generation) to better align patch evaluations with human judgments. Collectively, these studies highlight the expanding role of LLM-as-a-Judge frameworks not only in patch-level correctness evaluation but also in dataset curation and the establishment of reliable evaluation protocols for automated program repair and vulnerability mitigation.

⁶<https://www.openmp.org/>

⁷<https://www.openacc.org/>

Table 5. Overview on the application of LLM-as-a-Judge in Software Maintenance

	Evaluation Type	Evaluation Criteria	Evaluation Item	Evaluation Result
Zeng et al. [115]	Point-wise	Content Quality (What & Why)	Commit Message	Graded
Yadavally et al [103]	Point-wise	Functional Correctness, Functional	Code Patch	Graded
Li et al. [48]	Point-wise	Vulnerability Relevance	Code Patch	Graded
SWE-Judge [125]	Point-wise, Pair-wise	Functional Correctness, Functional Equivalence	Code Patch	Graded
Ahmed et al. [1]	Point-wise	Actionability (whether fixes the warning of static analyzers)	Code Patch	Graded

Table 6. Synthesis of empirical findings on the alignment between LLM and human evaluations across diverse SE artifacts.

SE Artifact	Alignment	Key Observations
Requirements Documents	Low	Agreement is weak to non-existent. LLMs are significantly stricter than human evaluators, flagging many more requirements as flawed.
System Specifications	N/A	No human comparison available. High F1 (79–92%) on detecting non-fulfilled requirements, Performance degrading when specs are written in a more conversational style.
User Stories	High	Near-perfect agreement when LLMs are guided by detailed criteria (codebooks).
Generated Code	Moderate	Strong agreement on surface-level style, but frequently misjudges functional validity and subtle logical bugs.
Code Summaries	Low	LLMs exhibit strong bias toward verbose, step-by-step summaries.
Translated Code	Moderate	Good correlation with human scores when judging semantic equivalence.
Software Q&A	Low	Fails to distinguish critical from trivial errors, often penalizing minor omissions as heavily as severe inaccuracies.
Bug Reports	High	Strong alignment on evaluating the quality of bug titles and summaries.
Test Artifacts	High	Strong alignment on subjective attributes, such as readability and test coverage.
Vulnerability Explanations	High	Nearly identical preferences to humans for qualitative reasoning about software vulnerabilities.
Commit Messages	Moderate	Generally aligned, though LLMs show a slight bias favoring LLM-written messages.
Code Patches	High	Strong correlation with human judgment, occasionally surpassing inter-human agreement.

4.5 Key Takeaways

To provide a holistic view of the current state of the art, we synthesize the empirical findings from the reviewed literature regarding the reliability of LLM-as-a-Judge. Table 6 summarizes the degree of alignment between LLM evaluations and human judgment across the various software engineering artifacts discussed. It highlights the varying levels of success—ranging from high alignment in tasks like bug reporting and vulnerability explanation to lower alignment in complex tasks like requirements engineering—and outlines the key observations driving these results.

Comparative Analysis of Model Families. Recent studies [1, 11, 89, 99] indicate that GPT, Claude, and LLaMA models each offer distinct advantages when acting as automated judges for software engineering tasks. The GPT family, particularly GPT-4, sets the benchmark for complex reasoning and stability, often achieving the highest alignment with human evaluators but with high API cost. Anthropic’s Claude 3 models excel at capturing fine-grained details and adhering strictly to evaluation rubrics, with large context windows that make them ideal for analyzing extensive codebases. Meanwhile, Meta’s LLaMA 2 and specialized open-source derivatives like FLAME are rapidly closing the performance gap; while they often require fine-tuning or ensembling to match proprietary reliability, they offer critical advantages in data privacy, customizability, and low costs for scalable deployment.

5 The Road Ahead

This section outlines a research roadmap for achieving scalable, reliable, and effective LLM-as-a-Judge systems in software engineering. We begin by outlining the key limitations in the current landscape and then propose research opportunities and concrete actionable steps that can significantly broaden the applicability of LLM-as-a-Judge across diverse SE contexts, steering us toward our envisioned future.

The proposed research roadmap, visualized in Figure 2, charts a course for advancing LLM-as-a-Judge systems in software engineering by 2030. The roadmap is structured around two key phases:

- **Foundational Short-Term Goals:** The initial phase focuses on building a stable foundation for future research.
 - (1) **Develop Comprehensive Benchmarks and a Nuanced Evaluation Protocol:** This goal aims to overcome the scarcity of large-scale, human-annotated benchmarks and the limitations of current metrics. The objective is to create diverse benchmarks with high-quality expert annotations covering nuanced criteria like readability and helpfulness. The new protocol will account for subjectivity in SE tasks by capturing the full distribution of expert opinions and using distribution-aware metrics for assessment.
 - (2) **Conduct Comprehensive Empirical Evaluations and Bias Analysis:** This goal seeks to resolve inconsistent empirical findings in the field. The plan is to conduct large-scale, systematic evaluations of LLM-as-a-Judge systems across various SE tasks. These studies will analyze key design variables, such as prompting strategies and LLM configurations, and will include in-depth investigations into cognitive biases that cause flawed judgments.
- **Parallel Long-Term Tracks:** Building on this foundation, three parallel research tracks will advance LLM judges into sophisticated systems for the SE lifecycle.
 - (1) **Enhancing Internal Intelligence:** Research will focus on improving the SE expertise of LLMs by training them on high-quality, domain-specific data and capturing the procedural knowledge of human experts. Another key advancement would be the integration of multi-modal assessment, enabling MLLM judges to evaluate visual artifacts like UML diagrams and GUI mockups.
 - (2) **Augmenting with External Insights:** This focuses on creating frameworks that integrate the LLM judge with a diverse set of external SE tools, such as formal verification systems. It also aims to establish a synergistic relationship between LLM judges and human experts, creating a continuous feedback loop where human corrections are used to refine the LLM judge.
 - (3) **Building Robust and Secure Judges:** This track focuses on systematically testing for vulnerabilities and strengthening defenses against advanced adversarial threats. Research will move beyond simple attacks to uncover vulnerabilities through advanced adversarial testing, enabling the development of resilient judges fortified against sophisticated, semantics-preserving manipulations.

The temporal structure of this roadmap reflects a critical technical dependency and increasing complexity: effective optimization necessitates accurate evaluation. As highlighted in Section 5.1, the current landscape is fragmented by inconsistent findings and a lack of ground truth. Consequently, our Foundational Short-Term Goals prioritize the establishment of rigorous benchmarks and protocols (standardization effort). Without them, progress on the Long-Term Development (methodological innovation) cannot be reliably validated.

5.1 Developing Larger-Scale Benchmarks and Rigorous Evaluation Frameworks

Limitation 1: Lack of Large-Scale High-Quality Human-Annotated Benchmarks. In Section 4, we reviewed numerous studies evaluating the effectiveness of LLM-as-a-Judge in software engineering [1, 11, 89, 90, 125]. This literature typically assesses the alignment between LLM-as-a-Judge and human judgment. However, a foundational limitation in the field is the scarcity of large-scale, human-annotated benchmarks for evaluating LLM-as-a-Judge systems. While established benchmarks such as HumanEval [10], HumanEval-XL [68], and CoNaLa [113] provide useful test cases for assessing the ability of LLM-as-a-Judge in evaluating code correctness,

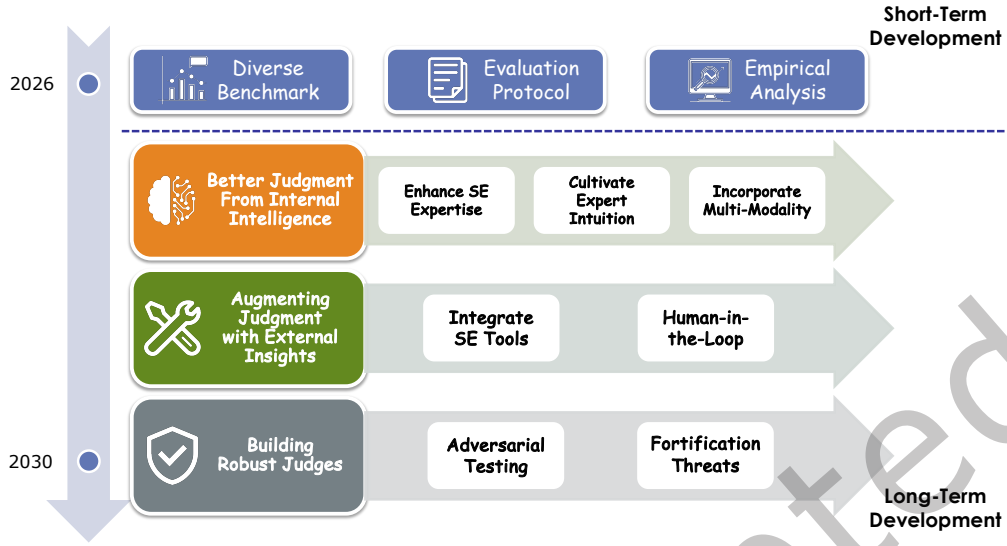


Fig. 2. The Roadmap for LLM-as-a-Judge in SE.

they fall short when assessing more nuanced aspects like helpfulness, readability, and alignment with human judgment. Consequently, a critical limitation in existing literature on LLM-as-a-Judge of SE is their reliance on small-scale datasets to measure human alignment [89, 99], often comprising only a few hundred samples. For example, Wang et al. [89] conducted experiments on three SE tasks, i.e., Code Translation, Code Generation, and Code Summarization, using a total of 450 samples. Similarly, Ahmed et al. [1] evaluated code summarization using 420 samples. Such limited datasets threaten the generalizability of findings and contribute to inconsistent empirical results across studies. CodeVisionary [90] conducted experiments on 363 human-annotated code generation samples. This scarcity is not merely an issue of effort; it points to a more fundamental difficulty in defining a “gold standard” for these subjective tasks.

Limitation 2: Neglect of Indeterminacy, Uncertainty, and Preferences. However, a high-quality benchmark for evaluating LLM-as-a-Judge in SE is indeed challenging to develop. A significant limitation in current SE benchmarks of LLM-as-a-Judge systems is the neglect of three intertwined concepts: *Rating Indeterminacy*, *Evaluator Uncertainty*, and *Evaluator Preferences*. Current validation practices often operate on the flawed assumption that a single, objective ground truth exists for subjective software engineering tasks. This oversimplification ignores that genuine expert evaluation is not a monolithic process, but is instead shaped by three critical factors: *Rating Indeterminacy*, *Evaluator Uncertainty*, and *Evaluator Preferences*.

- **Rating Indeterminacy:** A task is indeterminate [67] when it lacks a single, objectively correct rating. Indeterminacy allows for multiple reasonable assessments, even from one SE expert. This is not a flaw in the evaluation process but an inherent characteristic of many evaluation qualities in software engineering tasks. This property arises from two primary sources: vagueness and ambiguity. Vagueness stems from an imprecise definition of a criterion. For example, concepts like “a good design,” “maintainability,” or “readability,” lack a universally accepted definition. Ambiguity arises from a lack of sufficient context. However, current SE literature of LLM-as-a-Judge often obscures this reality by aggregating human ratings into a single “gold label,” thereby erasing legitimate alternative perspectives. The issue is exacerbated by **forced-choice elicitation**,

which requires evaluators to select only one option. As a result, this practice is likely to bias evaluations when humans and LLMs resolve indeterminacy differently.

- *Evaluator Uncertainty*: Evaluator uncertainty refers to the inherent variability in human judgments, where different experts may disagree on the same artifact. An indeterminate task often forces the judge to be uncertain.
- *Evaluator Preferences*: The issue of indeterminacy is further complicated by evaluator preferences. Preferences [53] are the specific, often unwritten, conventions of a team or individual regarding coding style, architectural patterns, documentation standards, etc. Indeterminacy creates the space for these preferences to become the deciding factor.

Limitation 3: Inadequacy of Traditional Alignment Metrics. Current SE literature of LLM-as-a-Judge often relies on metrics like inter-rater agreement (IRA) coefficients (e.g., Krippendorff’s α [41], Spearman’s ρ [96]) to measure the human-LLM alignment. However, these metrics are possibly ill-suited for comparing LLMs’ judgments to human evaluations because they were originally designed to measure agreement between human raters. The reasons are as follows:

- These metrics operate on the flawed premise that a single correct answer exists for subjective tasks like rating code readability. They penalize the natural variation in human perception by forcing researchers to aggregate diverse judgments into one “gold standard” label (e.g., a majority vote), discarding valuable information about the distribution of expert opinion.
- Metrics like Krippendorff’s α [41] were originally designed to measure the reliability of human labeling, and they assume all raters are interchangeable peers. This core assumption is clearly violated when comparing an LLM to human experts, as humans provide the ground truth labels.
- IRA metrics are designed to account for random, non-reproducible errors typical of human raters [16, 41, 96]. However, the errors made by LLMs are consistent and reproducible, even if hard to predict. Traditional metrics are ill-equipped to detect or correctly analyze this kind of systematic divergence between a human and LLMs.
- These IRA metrics do not provide a clear threshold for what constitutes a good performance. A Spearman’s correlation of 0.5 might be considered moderate, but it doesn’t tell a researcher if an LLM is a valid replacement for a human evaluator or what specific gaps in judgment exist.

Limitation 4: Inconsistent Empirical Findings. Consequently, the field is currently hampered by inconsistent and contradictory empirical findings [89, 99], which is a direct symptom of the previously discussed limitations. A few works reached varying conclusions due to discrepancies in dataset selection, prompting strategies, and LLM configurations. For instance, Wang et al. [89] found that traditional evaluation metrics (e.g., ROUGE [50], METEOR [6], and ChrF++ [69]) significantly outperformed LLM-as-a-Judge methods when aligning with human judgment for code summarization. In contrast, Wu et al. [99] reported that the LLM-as-a-Judge method surpassed conventional metrics in code summarization. These conflicting results highlight the need for a standardized, large-scale empirical study to enable fair and meaningful comparisons across studies.

Limitation 5: Limited Empirical Findings on Bias of LLM-as-a-Judge systems in SE. The evaluation of software artifacts by LLMs is susceptible to biases [95, 107]. LLMs are known to exhibit various cognitive biases, including Position Bias, where responses are favored based on their order [49]; Verbosity Bias, a preference for longer answers regardless of quality [37]; and Egocentric Bias, the tendency to overrate AI-generated solutions [112]. The SE community has started to investigate these cognitive biases. For instance, Crupi et al. [11] analyzed “self-bias,” where an LLM judge prefers its own generated content. However, existing work on LLM-as-a-Judge biases in SE is still in its early stages and overlooks more complex biases, such as Auxiliary Information Induced Biases [47]. These biases arise when an LLM judge is provided with external context, like reference answers or scoring rubrics. While intended to improve accuracy, this information can paradoxically make the judge vulnerable to new forms of bias and manipulation.

Opportunity: Develop Comprehensive and Diverse Benchmarks. Future research should work on creating large-scale, multi-dimensional benchmarks that capture the complexity of real-world software engineering tasks. Key steps include:

- (1) *Expert Annotations and Quality Control:* Engage a diverse pool of expert programmers to provide high-quality annotations. This annotation process should focus more on the nuanced criteria beyond code correctness, such as readability, helpfulness, and maintainability. In the beginning, define clear annotation guidelines that describe these nuanced dimensions to eliminate ambiguity. During the process, cross-validation and consensus mechanisms should be conducted to ensure the reliability of the annotations and improve the clarity of the guidelines.
- (2) *Dataset Expansion:* Expand existing datasets to include a larger volume of high-quality expert-annotated samples. These benchmarks should encompass SE tasks at varying difficulty levels, various programming topics, and multiple programming languages. Additionally, new benchmarks should be extended to evaluate a broader spectrum of SE artifacts. For instance, expert evaluations could be gathered for software requirements documentation, system design, and API documentation.
- (3) *Updating Human Preferences:* In the long term, the annotation process should be enriched to include explicit human preferences to reflect evolving SE practices.

Opportunity: Establish a Standard and Nuanced Evaluation Protocol. To overcome the current challenges, the community must move beyond simplistic aggregate scores and adopt a more rigorous evaluation paradigm. This requires establishing a standard protocol designed to handle the inherent subjectivity of software engineering tasks and provide a clearer, more comparable picture of an LLM's performance. Such a protocol should be built on the following principles:

- *Acknowledge the Subjectivity and Uncertainty:* Instead of aggregating diverse human ratings into a single "gold label," the protocol should mandate that benchmarks capture the full distribution of expert opinions. This paradigm shift treats the range of human judgments not as noise, but as the ground truth itself. This approach naturally accommodates tasks with inherent indeterminacy (e.g., rating "readability") and makes evaluator preferences an explicit part of the evaluation landscape rather than an unacknowledged variable.
- *Adopt Distribution-Aware Alignment Metrics:* The protocol must advocate for metrics that compare the distribution of LLM judgments against the distribution of human judgments. This moves away from metrics like Krippendorff's α or Spearman's ρ , which fundamentally rely on a single correct answer. Instead, research should focus on adopting or developing metrics (e.g., [58], Earth Mover's Distance [75]) that quantify how well an LLM's range of assessments mirrors the range of human expert assessments. This correctly reframes the goal from matching a single label to replicating the spectrum of expert reasoning.
- The protocol should require researchers to analyze and report performance separately on data subsets with high and low human agreement. By doing so, the community can distinguish an LLM's ability to handle clear, unambiguous cases from its performance on more contentious, indeterminate ones. This practice will expose the performance gaps that single aggregate scores often hide and help explain why different studies, using datasets with varying levels of uncertainty, have reached contradictory conclusions.

Opportunity: Conduct Comprehensive Empirical Evaluations in SE. Ultimately, the benchmarks and new evaluation protocol will lay a solid foundation for understanding the strengths and weaknesses of LLM-as-a-Judge systems in SE. The community will be positioned to conduct a large-scale, comprehensive empirical evaluation. The goal is to create a reliable, evidence-based foundation that provides clear and actionable guidance for both researchers and practitioners. This comprehensive evaluation should consider the following three core pillars:

- (1) *Systematic Comparison Across Diverse SE Tasks.* The study must systematically assess the performance of LLM-as-a-Judge across a wide array of software engineering tasks. This would go beyond conventional

tasks like code generation to include evaluations of software design documents, API usability, and system requirements. This broad scope will help identify where LLM judges excel and where they still fall short.

- (2) *Rigorous Analysis of Design Variables.* Studies should rigorously investigate how different design choices impact an LLM’s judgment. Key variables to analyze include:
 - *Prompting Strategies:* Evaluating the effectiveness of different prompting techniques, such as Chain-of-Thought, reference-based rubrics, and few-shot examples.
 - *LLM Configurations:* Comparing various models (e.g., GPT-4, Claude 3, Llama 3), model sizes, and parameters like temperature to understand their effect on evaluation quality and consistency.
- (3) *More Comprehensive Studies on Biases in LLM-as-a-Judge in SE.* Future research should conduct comprehensive empirical studies to better understand more complex biases in LLM-as-a-Judge systems for SE artifacts. For example, the auxiliary information-induced biases, where external context paradoxically leads to flawed evaluations. For example, studies can systematically investigate reference-induced biases, such as “solution fixation,” where a judge penalizes valid solutions that deviate from a given reference. Furthermore, research should explore whether the severity of these biases correlates with task complexity and whether advanced reasoning models are more susceptible than general-purpose models, particularly when their judgments rely on flawed auxiliary information.
- (4) *Creation of Actionable Guidance and Baselines.* The ultimate output should not just be a collection of results but a set of clear, actionable guidelines for the community. This would include recommendations on which models and prompting strategies are best suited for specific SE tasks, as well as establishing strong, reproducible baselines. This will enable future research to build on a common foundation, ensuring that new findings are comparable and contribute to a cumulative body of knowledge.

5.2 Better Judgment From Internal Intelligence

Limitation 6: Inadequate SE Domain-Specific Expertise in LLMs. While ideally, human evaluations should be conducted by experienced developers. Like human evaluators, LLM must exhibit a deep understanding of the task and the relevant domain knowledge to provide an accurate evaluation. Recent research highlights that LLMs have remarkable coding abilities, which builds confidence in their ability to assess tasks like code summarization and generation. However, research also indicates that some complex coding tasks are still challenging for LLMs [119, 131]. For instance, Zhao et al. [119] observed that current LLMs cannot generate a complete library from scratch, which also implies LLMs’ ability to evaluate library quality is still limited. Similar limitations extend to other critical areas in software engineering, such as software design, formal verification, distributed systems debugging, etc. Beyond gaps in SE-specific knowledge, LLMs may also struggle with discerning subtle differences among alternative solutions. As Zhao et al. [120] highlighted, an LLM’s ability to generate correct code does not guarantee that it can effectively evaluate alternative implementations for the same task.

Limitation 7: The Mono-modal Focus of Current LLM-as-a-Judge Evaluation. Current LLM-as-a-Judge research in SE is overwhelmingly mono-modal. It focuses almost exclusively on artifacts that exist as text and source code. This creates a significant blind spot, as software engineering is an inherently multi-modal discipline. Developers constantly work with visual artifacts like UML diagrams, architectural sketches, GUI mockups, and screenshots in bug reports. Current LLM judges cannot “see” these artifacts; they must rely on “flattened” textual descriptions, a process which can lose critical and structural information that is immediately apparent to a human expert.

Opportunity: Enhance the SE Expertise of LLMs. Future research can strengthen the SE domain-specific expertise of LLMs. A key approach is training LLMs on richer and more high-quality SE datasets [55]. Enhancing SE expertise in LLMs offers several benefits. For instance, LLMs with stronger formal verification capabilities

can more accurately assess code correctness. Further, research can expand LLM-as-a-Judge’s applicability to a broader range of SE artifacts, such as Docker files, UML diagrams, and formal specifications.

Opportunity: Embedding Expert Tacit Knowledge. Another critical research direction for enhancing judgment involves capturing the tacit and procedural knowledge that human experts develop over years of experience, i.e., their intuition and unspoken decision-making processes. We can employ methods such as structured interviews [77], think-aloud protocols [117], and cognitive task analyses [76], where experts are asked to articulate their reasoning while conducting real-world evaluations. This approach enables the systematic extraction of procedural knowledge that is typically not documented in software artifacts. Once captured, this knowledge can be integrated into LLM training through techniques like reinforcement learning [5] or neuro-symbolic methods [43].

Opportunity: Incorporation of Multi-modal Assessment. Multi-modal LLMs (MLLMs) [70, 98] are capable of reasoning over images, text, and code simultaneously, which presents a significant opportunity for the LLM-as-a-Judge paradigm [9]. The development of MLLM judges can enable holistic evaluations that more closely mimic the multi-modal workflow of an expert developer. MLLM judges can enable a set of cross-modal verifications that are currently beyond the capabilities of text-only models. For example:

- *Requirements-to-Design Conformance:* Verifying that a system’s design, as captured in a UML diagram or GUI mockup, faithfully implements the specified textual requirements.
- *Architectural Drift Detection:* Analyzing a high-level architectural diagram and comparing it against the code’s dependency graph to identify deviations from the intended architecture.
- *GUI Implementation and Usability Testing:* Evaluating a user interface by comparing application screenshots against design mockups to verify the precise implementation of visual elements and assess adherence to established UI/UX principles.

5.3 Augmenting Judgment with External Insights

Limitation 8: Limited Integration of External SE Tools. While early LLM-as-a-Judge methods depended exclusively on the model’s internal knowledge, this approach is insufficient for comprehensive evaluation, as it lacks access to crucial runtime, linting, and visual information. Pioneering research has begun to address this by developing agent-based frameworks [90, 128], like CodeVisionary [90], which can interact with and leverage external tools such as static linters, execution environments, and unit testers. However, this practice is still in its early stages and not yet widespread. The full potential of a synergistic relationship between the LLM judge and the rich ecosystem of specialized software engineering tools remains largely untapped.

Opportunity: Integrate Diverse SE Tools for Richer Evaluations. Building on the initial success of tool-augmented agents, a significant opportunity lies in creating frameworks where the LLM judge acts as an orchestrator, intelligently leveraging a wider array of external SE tools to form its judgments. While pioneering work like CodeVisionary has already integrated foundational tools like static linters and execution environments, future research can expand this integration to be far more comprehensive and sophisticated. For instance, the evaluation pipeline could incorporate formal verification frameworks [78], model checkers [12], and performance profilers to assess code efficiency, including execution time and memory usage [79, 105].

Opportunity: Intelligent Human-in-the-Loop Collaboration. For tasks that cannot be reliably automated by LLMs, we need to develop collaborative and interactive methods to include human oversight. A key component of such a system is enhancing LLMs with the ability to assess their own confidence. When a judge generates a low-confidence rating for a nuanced or high-stakes task, it should automatically flag the case for human review. This creates a structured workflow where LLMs handle high-volume, routine checks (e.g., style, simple correctness), allowing human experts to focus their limited time on the most complex evaluations. Also, identifying the optimal

ratio of LLM evaluators to human experts remains another challenge. Furthermore, this collaboration should not be a one-way street. The judgments and corrections provided by human experts must be collected as high-quality data to continuously fine-tune the LLM judge. This establishes a virtuous feedback loop, progressively aligning the AI's reasoning with that of human experts over time and addressing the challenge of finding the optimal balance between automated and manual evaluation.

5.4 Building Robust Judges

Limitation 9: Insufficient Research on Adversarial Threats and Defensive Methods in SE. Adversarial attacks on LLM-as-a-Judge systems pose a significant threat to software integrity. Current research in this area is still nascent. While frameworks like RobustJudge [46] have initiated the study of adversarial attacks in SE, their focus has largely been on overt methods such as prompt injection [46]. These attacks, often considered “low-hanging fruit,” are typically detectable and can be mitigated with straightforward defenses like input sanitization. However, this emphasis on overt attacks overlooks the more dangerous class of sophisticated, semantics-preserving manipulations. These attacks subtly alter software artifacts to deceive the judge into accepting flawed or malicious code [109, 110]. For instance, an attacker might introduce convoluted but functionally correct logic that harms maintainability, or using intentionally confusing variable names to mislead the judge. Because such subtle manipulations mimic legitimate coding practices, they are far more difficult to detect automatically. Consequently, this leaves LLM judges vulnerable to advanced threats that have serious implications for code quality and security.

Opportunity: Uncovering Vulnerabilities via Advanced Adversarial Testing. The nascent state of security research presents a critical opportunity to move beyond testing for overt attacks and systematically evaluate the robustness of LLM judges against more sophisticated threats. Future work should focus on developing and applying advanced adversarial testing methods to uncover hidden vulnerabilities. This includes creating optimization-based attacks, which use automated refinement to craft subtle inputs that probe for weaknesses in a model's reasoning. Another critical area for adversarial testing is evaluating resilience against persistent backdoor attacks [23, 28, 111], where a judge's training data is poisoned with innocuous triggers. For example, testing could involve inserting a specific comment style as a trigger to see if it activates a biased judgment. Such testing is crucial because, unlike overt attacks, these vulnerabilities are stealthy and bypass standard defenses, posing a persistent threat. A rigorous adversarial testing framework is essential for understanding these complex failure modes before effective defenses can be developed.

Opportunity: Fortifying Judges Against Advanced Threats. Future research should focus on building robust LLM judges capable of withstanding sophisticated adversarial threats. This requires a two-pronged strategy that combines proactive defense mechanisms with efforts to enhance the model's intrinsic security. On the one hand, researchers must develop defenses against advanced attacks like optimization-based methods and persistent backdoors. This involves creating dynamic defense mechanisms, such as anomaly detection [82, 111], and new techniques for data sanitization and model analysis to neutralize stealthy vulnerabilities introduced through data poisoning. On the other hand, a more powerful approach is to enhance the judge's intrinsic robustness, reducing reliance on external defenses. A promising direction is adversarial fine-tuning, where models are explicitly trained on datasets of successful attacks [24, 100, 108] or on pairs of adversarial and “purified” code to learn to identify and resist manipulation patterns. Furthermore, advanced prompting techniques can foster self-awareness and skepticism in the judge. For instance, incorporating a Chain-of-Thought for security analysis or implementing self-correction cycles would enable the model to critique its own judgments for potential adversarial influence before delivering a final evaluation. By integrating both external defenses and intrinsic hardening, the community can develop more resilient and trustworthy LLM-as-a-Judge systems.

Figure 3 visualizes the direct correspondence between the current bottlenecks and our proposed solutions. This mapping explicitly links each identified limitation (L1–L9) to specific strategic opportunities.

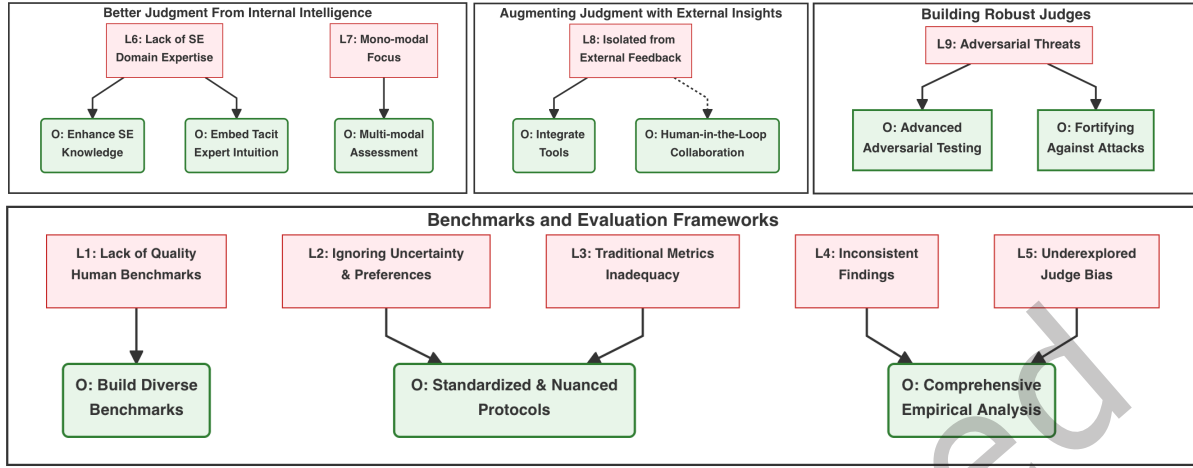


Fig. 3. A structural overview of the roadmap connecting specific limitations (L) to their corresponding strategic opportunities (O).

6 A Vision for the Future

This section presents our vision for LLM-as-a-Judge, illustrating the future capabilities outlined in our roadmap for evaluating LLM-generated software artifacts.

6.1 Calibrated Expert Judgment

Beyond assessing functional correctness, the LLM Judge will deliver nuanced verdicts that emulate the assessments of an expert human developer. It will serve as a cost-effective surrogate for human experts, capable of evaluating complex qualities such as code maintainability and the presence of code smells. A significant leap will be in the domain of security; while current models often lack the precision for reliable vulnerability detection, the future LLM Judge will provide accurate judgments in this critical area. This expert-level feedback can be provided in real-time, enabling the rapid and iterative refinement of generated artifacts at scale.

6.2 Preference-Aware, Multi-Perspective Feedback

The visionary LLM Judge will treat subjectivity as a configurable feature. By adopting distinct evaluation personas, it will emulate the preferences of different domain experts and assess an artifact from multiple valid viewpoints. This approach directly addresses the challenges of Rating Indeterminacy and Evaluator Preferences. Such targeted feedback allows evaluation to be tailored to specific project needs, overcoming the limitations of a single, generic “gold standard”.

6.3 Cross-Modal Holistic Assessment

Software engineering is an inherently multi-modal discipline. By 2030, the LLM Judge will be a Multi-modal LLM, capable of reasoning across diverse artifact types to ensure their coherence. For example, an MLLM Judge could simultaneously analyze a generated UML class diagram and its corresponding code implementation to identify discrepancies between the design and its implementation. This cross-modal verification, impossible for current text-only models, represents a major leap toward achieving holistic and reliable evaluation.

Ultimately, this visionary judge will provide a comprehensive, multi-faceted, and adaptable evaluation. It will ensure the quality, security, performance, and internal consistency of LLM-generated software, far exceeding the capabilities of today's automated metrics.

7 Conclusion

This forward-looking SE 2030 paper presents an overview of the current landscape of LLM-as-a-Judge systems in software engineering. We conduct a systematic review of 42 primary studies in the field, and map the applications across the software development lifecycle. Further, we identify the limitations of LLM-as-a-Judge in SE while highlighting the associated opportunities and presenting a future vision for its adoption in the community. To realize this vision by 2030 and beyond, we propose a research roadmap outlining the specific paths and potential solutions that the research community can pursue. We propose a list of short-term goals and long-term goals. Through these efforts, we aim to encourage broader participation in the LLM-as-a-Judge research journey.

Acknowledgments

This research / project is supported by the National Research Foundation, under its Investigatorship Grant (NRF-NRFI08-2022-0002). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore.

References

- [1] Toufique Ahmed, Premkumar Devanbu, Christoph Treude, and Michael Pradel. 2025. Can LLMs Replace Manual Annotation of Software Engineering Artifacts? . In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE Computer Society, Los Alamitos, CA, USA, 526–538. doi:10.1109/MSR66628.2025.00086
- [2] Toufique Ahmed, Kunal Suresh Pai, Premkumar Devanbu, and Earl Barr. 2024. Automatic semantic augmentation of language model prompts (for code summarization). In *Proceedings of the IEEE/ACM 46th international conference on software engineering*. 1–13.
- [3] Anthropic. 2023. Claude (Oct 8 version) [Large language model]. <https://www.anthropic.com/>
- [4] Anysphere. 2025. Cursor - The AI Code Editor – cursor.com. <https://www.cursor.com>.
- [5] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. 2022. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862* (2022).
- [6] Satanjeev Banerjee and Alon Lavie. 2005. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In *Proceedings of the acl workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization*. 65–72.
- [7] Cameron B Browne, Edward Powley, Daniel Whitehouse, Simon M Lucas, Peter I Cowling, Philipp Bohnlshagen, Stephen Tavenor, Diego Perez, Spyridon Samothrakis, and Simon Colton. 2012. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in games* 4, 1 (2012), 1–43.
- [8] Raymond PL Buse, Caitlin Sadowski, and Westley Weimer. 2011. Benefits and barriers of user evaluation in software engineering research. In *Proceedings of the 2011 ACM international conference on Object oriented programming systems languages and applications*. 643–656.
- [9] Dongping Chen, Ruoxi Chen, Shilin Zhang, Yaochen Wang, Yinyao Liu, Huichi Zhou, Qihui Zhang, Yao Wan, Pan Zhou, and Lichao Sun. 2024. Mllm-as-a-judge: Assessing multimodal llm-as-a-judge with vision-language benchmark. In *Forty-first International Conference on Machine Learning*.
- [10] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [11] Giuseppe Crupi, Rosalia Tufano, Alejandro Velasco, Antonio Mastropaolo, Denys Poshyvanyk, and Gabriele Bavota. 2025. On the Effectiveness of LLM-as-a-judge for Code Generation and Summarization. *IEEE Transactions on Software Engineering* (2025).
- [12] LO David and KHOO Siau-Cheng. 2007. Model Checking in the Absence of Code, Model and Properties. In *5th Asian Symposium on Programming Languages and Systems (APLAS)(Poster Track)*, November 29-December 1.
- [13] Aixin Liu DeepSeek-AI, Bei Feng, Bin Wang, B Wang, B Liu, C Zhao, C Dengr, C Ruan, D Dai, D Guo, et al. [n. d.]. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024. URL <https://arxiv.org/abs/2405.04434> ([n. d.]).
- [14] Enrique Dehaerne, Bappaditya Dey, Sandip Halder, Stefan De Gendt, and Wannes Meert. 2022. Code generation using machine learning: A systematic review. *Ieee Access* 10 (2022), 82434–82455.

- [15] Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Weikang Zhou, Muling Wu, Mingxu Chai, Jessica Fan, Caishuang Huang, Yunbo Tao, et al. 2024. What's Wrong with Your Code Generated by Large Language Models? An Extensive Study. *arXiv preprint arXiv:2407.06153* (2024).
- [16] Aparna Elangovan, Lei Xu, Jongwoo Ko, Mahsa Elyasi, Ling Liu, Sravan Bodapati, and Dan Roth. 2024. Beyond correlation: The impact of human uncertainty in measuring the effectiveness of automatic evaluation and LLM-as-a-judge. *arXiv preprint arXiv:2410.03775* (2024).
- [17] Mikhail Evtikhiev, Egor Bogomolov, Yaroslav Sokolov, and Timofey Bryksin. 2023. Out of the bleu: how should we assess quality of the code generation models? *Journal of Systems and Software* 203 (2023), 111741.
- [18] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. 31–53. doi:10.1109/ICSE-FoSE59343.2023.00008
- [19] Eitan Farchi, Shmulik Froimovich, Rami Katan, and Orna Raz. 2024. Automatic Generation of Benchmarks and Reliable LLM Judgment for Code Tasks. *arXiv preprint arXiv:2410.21071* (2024).
- [20] Arduin Findeis, Floris Weers, Guoli Yin, Ke Ye, Ruoming Pang, and Tom Gunter. 2025. Can External Validation Tools Improve Annotation Quality for LLM-as-a-Judge? *arXiv preprint arXiv:2507.17015* (2025).
- [21] Cuiyun Gao, Xing Hu, Shan Gao, Xin Xia, and Zhi Jin. 2025. The Current Challenges of Software Engineering in the Era of Large Language Models. *ACM Trans. Softw. Eng. Methodol.* (Jan. 2025). doi:10.1145/3712005 Just Accepted.
- [22] GitHub. 2025. GitHub Copilot · Your AI pair programmer. <https://github.com/features/copilot/>.
- [23] Chen Gong, Zhou Yang, Yunpeng Bai, Junda He, Jieke Shi, Kecen Li, Arunesh Sinha, Bowen Xu, Xinwen Hou, David Lo, et al. 2024. Baffle: Hiding backdoors in offline reinforcement learning datasets. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2086–2104.
- [24] Chen Gong, Zhou Yang, Yunpeng Bai, Jieke Shi, Arunesh Sinha, Bowen Xu, David Lo, Xinwen Hou, and Guoliang Fan. 2022. Curiosity-driven and victim-aware adversarial policies. In *Proceedings of the 38th Annual Computer Security Applications Conference*. 186–200.
- [25] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [26] Junda HE. 2026. Online Appendix for Paper: From Code to Courtroom: LLMs as the New Software Judges. doi:10.6084/m9.figshare.31044337
- [27] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* (2025).
- [28] Junda He, Zhou Yang, Jieke Shi, Chengran Yang, Kisub Kim, Bowen Xu, Xin Zhou, and David Lo. 2024. Curiosity-driven testing for sequential decision-making process. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–14.
- [29] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. 2021. Measuring coding challenge competence with apps (2021). *arXiv preprint arXiv:2105.09938* (2021).
- [30] Xing Hu, Qiuyuan Chen, Haoye Wang, Xin Xia, David Lo, and Thomas Zimmermann. 2022. Correlating automated and human evaluation of code documentation generation quality. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 4 (2022), 1–28.
- [31] Xing Hu, Feifei Niu, Junkai Chen, Xin Zhou, Junwei Zhang, Junda He, Xin Xia, and David Lo. 2025. Assessing and Advancing Benchmarks for Evaluating Large Language Models in Software Engineering Tasks. *arXiv preprint arXiv:2505.08903* (2025).
- [32] Anna A Ivanova. 2025. How to evaluate the cognitive abilities of LLMs. *Nature Human Behaviour* (2025), 1–4.
- [33] Samireh Jalali and Claes Wohlin. 2012. Systematic literature studies: database searches vs. backward snowballing. In *Proceedings of the ACM-IEEE international symposium on Empirical software engineering and measurement*. 29–38.
- [34] Imen Jaoua, Oussama Ben Sghaier, and Houari Sahraoui. 2025. Combining Large Language Models with Static Analyzers for Code Review Generation. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 174–186.
- [35] Hongchao Jiang, Yiming Chen, Yushi Cao, Hung-yi Lee, and Robby T Tan. 2025. CodeJudgeBench: Benchmarking LLM-as-a-Judge for Coding Tasks. *arXiv preprint arXiv:2507.10535* (2025).
- [36] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of Code Language Models on Automated Program Repair. In *Proceedings of the 45th International Conference on Software Engineering (Melbourne, Victoria, Australia) (ICSE '23)*. IEEE Press, 1430–1442. doi:10.1109/ICSE48619.2023.00125
- [37] Tong Jiao, Jian Zhang, Kui Xu, Rui Li, Xi Du, Shangqi Wang, and Zhenbo Song. 2024. Enhancing Fairness in LLM Evaluations: Unveiling and Mitigating Biases in Standard-Answer-Based Evaluations. In *Proceedings of the AAAI Symposium Series*, Vol. 4. 56–59.
- [38] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. InferFix: End-to-End Program Repair with LLMs. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (San Francisco, CA, USA) (ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 1646–1656. doi:10.1145/3611643.3613892

- [39] Gyeongwon James Kim, Alex Wilf, Louis-Philippe Morency, and Daniel Fried. 2025. From Reproduction to Replication: Evaluating Research Agents with Progressive Code Masking. *arXiv preprint arXiv:2506.19724* (2025).
- [40] Charles Koutchme, Nicola Dainese, Sami Sarsa, Arto Hellas, Juho Leinonen, Syed Ashraf, and Paul Denny. 2025. Evaluating language models for generating and judging programming feedback. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1*. 624–630.
- [41] Klaus Krippendorff. 2011. Computing Krippendorff’s alpha-reliability. (2011).
- [42] Abhishek Kumar, Sonia Haiduc, Partha Pratim Das, and Partha Pratim Chakrabarti. 2024. LLMs as Evaluators: A Novel Approach to Evaluate Bug Report Summarization. *arXiv preprint arXiv:2409.00630* (2024).
- [43] Joe Kwon, Sydney Levine, and Joshua B Tenenbaum. 2023. Neuro-symbolic models of human moral judgment: LLMs as automatic feature extractors. (2023).
- [44] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. 2024. LLMs-as-judges: a comprehensive survey on llm-based evaluation methods. *arXiv preprint arXiv:2412.05579* (2024).
- [45] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).
- [46] Songze Li, Chuokun Xu, Jiaying Wang, Xueluan Gong, Chen Chen, Jirui Zhang, Jun Wang, Kwok-Yan Lam, and Shouling Ji. 2025. LLMs Cannot Reliably Judge (Yet?): A Comprehensive Assessment on the Robustness of LLM-as-a-Judge. *arXiv preprint arXiv:2506.09443* (2025).
- [47] Weiyuan Li, Xintao Wang, Siyu Yuan, Rui Xu, Jiangjie Chen, Qingqing Dong, Yanghua Xiao, and Deqing Yang. 2025. Curse of Knowledge: When Complex Evaluation Context Benefits yet Biases LLM Judges. *arXiv preprint arXiv:2509.03419* (2025).
- [48] Yikun Li, Ting Zhang, Ratnadira Widyasari, Yan Naing Tun, Huu Hung Nguyen, Tan Bui, Ivana Clairine Irsan, Yiran Cheng, Xiang Lan, Han Wei Ang, et al. 2024. CleanVul: Automatic Function-Level Vulnerability Detection in Code Commits Using LLM Heuristics. *arXiv preprint arXiv:2411.17274* (2024).
- [49] Zongjie Li, Chaozheng Wang, Pingchuan Ma, Daoyuan Wu, Shuai Wang, Cuiyun Gao, and Yang Liu. 2024. Split and Merge: Aligning Position Biases in LLM-based Evaluators. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 11084–11108. doi:10.18653/v1/2024.emnlp-main.621
- [50] Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*. 74–81.
- [51] Aixun Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [52] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, Li Zhang, Zhongqi Li, and Yuchi Ma. 2024. Exploring and evaluating hallucinations in llm-powered code generation. *arXiv preprint arXiv:2404.00971* (2024).
- [53] Jiawei Liu, Thanh Nguyen, Mingyue Shang, Hantian Ding, Xiaopeng Li, Yu Yu, Varun Kumar, and Zijian Wang. 2024. Learning code preference via synthetic evolution. *arXiv preprint arXiv:2410.03837* (2024).
- [54] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2024. Large language model-based agents for software engineering: A survey. *arXiv preprint arXiv:2409.02977* (2024).
- [55] Yang Liu, Jiahuan Cao, Chongyu Liu, Kai Ding, and Lianwen Jin. 2024. Datasets for large language models: A comprehensive survey. *arXiv preprint arXiv:2402.18041* (2024).
- [56] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173* (2024).
- [57] Sebastian Lubos, Alexander Felfernig, Thi Ngoc Trang Tran, Damian Garber, Merfat El Mansi, Seda Polat Erdeniz, and Viet-Man Le. 2024. Leveraging llms for the quality assurance of software requirements. In *2024 IEEE 32nd International Requirements Engineering Conference (RE)*. IEEE, 389–397.
- [58] María Luisa Menéndez, Julio Angel Pardo, Leandro Pardo, and Maria del C Pardo. 1997. The jensen-shannon divergence. *Journal of the Franklin Institute* 334, 2 (1997), 307–318.
- [59] Jiwon Moon, Yerim Hwang, Dongryeol Lee, Taegwan Kang, Yongil Kim, and Kyomin Jung. 2025. Don’t Judge Code by Its Cover: Exploring Biases in LLM Judges for Code Evaluation. *arXiv preprint arXiv:2505.16222* (2025).
- [60] Wenhan Mu, Ling Xu, Shuren Pei, Le Mi, and Huichi Zhou. 2025. Evaluate-and-purify: Fortifying code language models against adversarial attacks using llm-as-a-judge. In *International Conference on Intelligent Computing*. Springer, 369–379.
- [61] OpenAI. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023). <https://arxiv.org/abs/2303.08774>
- [62] Jane Pan, Ryan Shar, Jacob Pfau, Ameet Talwalkar, He He, and Valerie Chen. 2025. When Benchmarks Talk: Re-Evaluating Code LLMs with Interactive Feedback. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 24672–24700. doi:10.18653/v1/2025.findings-acl.1267
- [63] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2023. Understanding the effectiveness of large language models in code translation.

- CoRR (2023).
- [64] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 311–318.
 - [65] Bhrij Patel, Souradip Chakraborty, Wesley A Suttle, Mengdi Wang, Amrit Singh Bedi, and Dinesh Manocha. 2024. AIME: AI System Optimization via Multiple LLM Evaluators. *arXiv preprint arXiv:2410.03131* (2024).
 - [66] Avinash Patil. 2025. Advancing Reasoning in Large Language Models: Promising Methods and Approaches. *arXiv preprint arXiv:2502.03671* (2025).
 - [67] Ellie Pavlick and Tom Kwiatkowski. 2019. Inherent disagreements in human textual inferences. *Transactions of the Association for Computational Linguistics* 7 (2019), 677–694.
 - [68] Qiwei Peng, Yekun Chai, and Xuhong Li. 2024. Humaneval-xl: A multilingual code generation benchmark for cross-lingual natural language generalization. *arXiv preprint arXiv:2402.16694* (2024).
 - [69] Maja Popović. 2017. chrF++: words helping character n-grams. In *Proceedings of the second conference on machine translation*. 612–618.
 - [70] Libo Qin, Qiguang Chen, Yuhang Zhou, Zhi Chen, Yinghui Li, Lizi Liao, Min Li, Wanxiang Che, and Philip S Yu. 2024. Multilingual large language model: A survey of resources, taxonomy and frontiers. *arXiv preprint arXiv:2404.04925* (2024).
 - [71] Giovanni Quattrocchi, Liliana Pasquale, Paola Spoletini, and Luciano Baresi. 2025. Can LLMs Generate User Stories and Assess Their Quality? *arXiv preprint arXiv:2507.15157* (2025).
 - [72] Lasse M Reinhold, Marvin Schiesack, Lukas P Wagner, Felix Gehlhoff, and Alexander Fay. 2024. Exploring LLMs for Verifying Technical System Specifications Against Requirements. In *2024 IEEE 3rd Industrial Electronics Society Annual On-Line Conference (ONCON)*. IEEE, 1–6.
 - [73] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* (2020).
 - [74] Devjeet Roy, Sarah Fakhoury, and Venera Arnaoudova. 2021. Reassessing automatic evaluation metrics for code summarization tasks. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1105–1116.
 - [75] Yossi Rubner, Carlo Tomasi, and Leonidas J Guibas. 2000. The earth mover’s distance as a metric for image retrieval. *International journal of computer vision* 40, 2 (2000), 99–121.
 - [76] Jan Maarten Schraagen, Susan F Chipman, and Valerie L Shalin. 2000. *Cognitive task analysis*. Psychology Press.
 - [77] Daniel L Segal, Frederick L Coolidge, Alisa O’Riley, and Benjamin A Heinz. 2006. Structured and semistructured interviews. In *Clinician’s handbook of adult behavioral assessment*. Elsevier, 121–144.
 - [78] Jieke Shi, Junda He, Zhou Yang, Đorđe Žikelić, and David Lo. 2025. Synthesizing Efficient and Permissive Programmatic Runtime Shields for Neural Policies. *arXiv:2410.05641 [cs.SE]* <https://arxiv.org/abs/2410.05641>
 - [79] Jieke Shi, Zhou Yang, and David Lo. 2025. Efficient and Green Large Language Models for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (2025), 137:1–137:22. doi:10.1145/3708525
 - [80] Igor Regis da Silva Simões and Elaine Venson. 2024. Evaluating Source Code Quality with Large Language Models: a comparative study. In *Proceedings of the XXIII Brazilian Symposium on Software Quality*. 103–113.
 - [81] Zachariah Sollenberger, Jay Patel, Christian Munley, Aaron Jarmusch, and Sunita Chandrasekaran. 2024. LLM4VV: Exploring LLM-as-a-Judge for Validation and Verification Testsuites. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1885–1893.
 - [82] Shuang Song, Yifei Zhang, and Neng Gao. 2025. Confront Insider Threat: Precise Anomaly Detection in Behavior Logs Based on LLM Fine-Tuning. In *Proceedings of the 31st International Conference on Computational Linguistics*, Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). Association for Computational Linguistics, Abu Dhabi, UAE, 8589–8601. <https://aclanthology.org/2025.coling-main.574/>
 - [83] Weisong Sun, Yun Miao, Yuekang Li, Hongyu Zhang, Chunrong Fang, Yi Liu, Gelei Deng, Yang Liu, and Zhenyu Chen. 2024. Source code summarization in the era of large language models. *arXiv preprint arXiv:2407.07959* (2024).
 - [84] Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. 2024. Judgebench: A benchmark for evaluating llm-based judges. *arXiv preprint arXiv:2410.12784* (2024).
 - [85] Weixi Tong and Tianyi Zhang. 2024. CodeJudge: Evaluating Code Generation with Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 20032–20051. doi:10.18653/v1/2024.emnlp-main.1118
 - [86] Ngoc Phuoc An Vo, Brent Paulovicks, and Vadim Sheinin. 2025. LLM-as-a-Judge for Reference-less Automatic Code Validation and Refinement for Natural Language to Bash in IT Automation. *arXiv preprint arXiv:2506.11237* (2025).
 - [87] Tu Vu, Kalpesh Krishna, Salaheddin Alzubi, Chris Tar, Manaal Faruqui, and Yun-Hsuan Sung. [n. d.]. Foundational autoraters: Taming large language models for better automatic evaluation, 2024. URL <https://arxiv.org/abs/2407.10817> ([n. d.]).
 - [88] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software Testing With Large Language Models: Survey, Landscape, and Vision. *IEEE Transactions on Software Engineering* 50, 4 (2024), 911–936. doi:10.1109/TSE.2024.3368208

- [89] Ruiqi Wang, Jiyu Guo, Cuiyun Gao, Guodong Fan, Chun Yong Chong, and Xin Xia. 2025. Can LLMs Replace Human Evaluators? An Empirical Study of LLM-as-a-Judge in Software Engineering. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA086 (June 2025), 23 pages. doi:10.1145/3728963
- [90] Xincheng Wang, Pengfei Gao, Chao Peng, Ruida Hu, and Cuiyun Gao. 2025. CodeVisionary: An Agent-based Framework for Evaluating Large Language Models in Code Generation. *arXiv preprint arXiv:2504.13472* (2025).
- [91] Yue Wang, Zhenyu Chen, Yuan Zhao, Chunrong Fang, Ziyuan Wang, and Song Huang. 2025. RUM: Rule+ LLM-Based Comprehensive Assessment on Testing Skills. *arXiv preprint arXiv:2508.12922* (2025).
- [92] Yutong Wang, Pengliang Ji, Chaoqun Yang, Kaixin Li, Ming Hu, Jiaoyang Li, and Guillaume Sartoretti. 2025. Mcts-judge: Test-time scaling in llm-as-a-judge for code correctness evaluation. *arXiv preprint arXiv:2502.12468* (2025).
- [93] Martin Weyssow, Aton Kamanda, Xin Zhou, and Houari Sahraoui. 2024. Codeultrafeedback: An llm-as-a-judge dataset for aligning large language models to coding preferences. *arXiv preprint arXiv:2403.09032* (2024).
- [94] Martin Weyssow, Chengran Yang, Junkai Chen, Yikun Li, Huihui Huang, Ratnadira Widyasari, Han Wei Ang, Frank Liauw, Eng Lieh Ouh, Lwin Khin Shar, et al. 2025. R2Vul: Learning to Reason about Software Vulnerabilities with Reinforcement Learning and Structured Reasoning Distillation. *arXiv preprint arXiv:2504.04699* (2025).
- [95] Ratnadira Widyasari, Stefanus Agus Haryono, Ferdian Thung, Jieke Shi, Constance Tan, Fiona Wee, Jack Phan, and David Lo. 2022. On the Influence of Biases in Bug Localization: Evaluation and Benchmark. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 128–139. doi:10.1109/SANER53432.2022.00027
- [96] Clark Wissler. 1905. The Spearman correlation formula. *Science* 22, 558 (1905), 309–311.
- [97] Claes Wohlin. 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th international conference on evaluation and assessment in software engineering*. 1–10.
- [98] Jiannan Wu, Muyan Zhong, Sen Xing, Zeqiang Lai, Zhaoyang Liu, Zhe Chen, Wenhai Wang, Xizhou Zhu, Lewei Lu, Tong Lu, et al. 2024. Visionllm v2: An end-to-end generalist multimodal large language model for hundreds of vision-language tasks. *Advances in Neural Information Processing Systems* 37 (2024), 69925–69975.
- [99] Yang Wu, Yao Wan, Zhaoyang Chu, Wenting Zhao, Ye Liu, Hongyu Zhang, Xuanhua Shi, and Philip S Yu. 2024. Can Large Language Models Serve as Evaluators for Code Summarization? *arXiv preprint arXiv:2412.01333* (2024).
- [100] Sophie Xhonneux, Alessandro Sordani, Stephan Günnemann, Gauthier Gidel, and Leo Schwinn. 2024. Efficient Adversarial Training in LLMs with Continuous Attacks. arXiv:2405.15589 [cs.LG] <https://arxiv.org/abs/2405.15589>
- [101] Wenwen Xie, Gray Gwizdz, and Dongji Feng. 2025. Prompting a Weighting Mechanism into LLM-as-a-Judge in Two-Step: A Case Study. *arXiv preprint arXiv:2502.13396* (2025).
- [102] Fangzhou Xu, Sai Zhang, Zhenchang Xing, Xiaowang Zhang, Yahong Han, and Zhiyong Feng. 2024. Human-Like Code Quality Evaluation through LLM-based Recursive Semantic Comprehension. *arXiv preprint arXiv:2412.00314* (2024).
- [103] Aashish Yadavally, Hoan Nguyen, Laurent Callot, and Gauthier Guinet. 2025. Large Language Model Critics for Execution-Free Evaluation of Code Changes. *arXiv preprint arXiv:2501.16655* (2025).
- [104] Weixiang Yan, Yuchen Tian, Yunzhe Li, Qian Chen, and Wen Wang. [n. d.]. CodeTransOcean: a comprehensive multilingual benchmark for code translation (2023). URL <https://arxiv.org/abs/2310.4951> ([n. d.]).
- [105] Chengran Yang, Hong Jin Kang, Jieke Shi, and David Lo. 2024. ACECode: A Reinforcement Learning Framework for Aligning Code Efficiency and Correctness in Code Language Models. arXiv:2412.17264 [cs.SE] <https://arxiv.org/abs/2412.17264>
- [106] Guang Yang, Yu Zhou, Xiang Chen, Wei Zheng, Xing Hu, Xin Zhou, David Lo, and Taolue Chen. 2025. CODE-DITING: A Reasoning-Based Metric for Functional Alignment in Code Evaluation. *arXiv preprint arXiv:2505.19502* (2025).
- [107] Zhou Yang, Harshit Jain, Jieke Shi, Muhammad Hilmi Asyrofi, and David Lo. 2021. BiasHeal: On-the-Fly Black-Box Healing of Bias in Sentiment Analysis Systems. In *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 644–648. doi:10.1109/ICSME52107.2021.00073
- [108] Zhou Yang, Jieke Shi, Muhammad Hilmi Asyrofi, and David Lo. 2022. Revisiting Neuron Coverage Metrics and Quality of Deep Neural Networks. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 408–419. doi:10.1109/SANER53432.2022.00056
- [109] Zhou Yang, Jieke Shi, Junda He, and David Lo. 2022. Natural attack for pre-trained models of code. In *Proceedings of the 44th International Conference on Software Engineering*. 1482–1493.
- [110] Zhou Yang, Zhensu Sun, Terry Zhuo Yue, Premkumar Devanbu, and David Lo. 2024. Robustness, security, privacy, explainability, efficiency, and usability of large language models for code. *arXiv preprint arXiv:2403.07506* (2024).
- [111] Zhou Yang, Bowen Xu, Jie M. Zhang, Hong Jin Kang, Jieke Shi, Junda He, and David Lo. 2024. Stealthy Backdoor Attack for Code Models. *IEEE Transactions on Software Engineering* 50, 4 (2024), 721–741. doi:10.1109/TSE.2024.3361661
- [112] Jiayi Ye, Yanbo Wang, Yue Huang, Dongping Chen, Qihui Zhang, Nuno Moniz, Tian Gao, Werner Geyer, Chao Huang, Pin-Yu Chen, et al. 2024. Justice or prejudice? quantifying biases in llm-as-a-judge. *arXiv preprint arXiv:2410.02736* (2024).
- [113] Pengcheng Yin, Bowen Deng, Edgar Chen, Bogdan Vasilescu, and Graham Neubig. 2018. Learning to mine aligned code and natural language pairs from stack overflow. In *Proceedings of the 15th international conference on mining software repositories*. 476–486.

- [114] Zhiqiang Yuan, Weitong Chen, Hanlin Wang, Kai Yu, Xin Peng, and Yiling Lou. 2024. Transagent: An llm-based multi-agent system for code translation. *arXiv preprint arXiv:2409.19894* (2024).
- [115] Qunhong Zeng, Yuxia Zhang, Zexiong Ma, Bo Jiang, Ningyuan Sun, Klaas-Jan Stol, Xingyu Mou, and Hui Liu. 2025. Evaluating Generated Commit Messages with Large Language Models. *arXiv preprint arXiv:2507.10906* (2025).
- [116] Jie M Zhang, Mark Harman, Lei Ma, and Yang Liu. 2020. Machine learning testing: Survey, landscapes and horizons. *IEEE Transactions on Software Engineering* 48, 1 (2020), 1–36.
- [117] Lawrence Jun Zhang and Donglan Zhang. 2019. Think-aloud protocols. In *The Routledge handbook of research methods in applied linguistics*. Routledge, 302–311.
- [118] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. 2019. Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675* (2019).
- [119] Wenting Zhao, Nan Jiang, Celine Lee, Justin T Chiu, Claire Cardie, Matthias Gallé, and Alexander M Rush. 2024. Commit0: Library Generation from Scratch. *arXiv preprint arXiv:2412.01769* (2024).
- [120] Yuwei Zhao, Ziyang Luo, Yuchen Tian, Hongzhan Lin, Weixiang Yan, Annan Li, and Jing Ma. 2025. CodeJudge-Eval: Can Large Language Models be Good Judges in Code Understanding?. In *Proceedings of the 31st International Conference on Computational Linguistics*, Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). Association for Computational Linguistics, Abu Dhabi, UAE, 73–95. <https://aclanthology.org/2025.coling-main.7/>
- [121] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2024. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems* 36 (2024).
- [122] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. 2023. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5673–5684.
- [123] Shuyan Zhou, Uri Alon, Sumit Agarwal, and Graham Neubig. 2023. Codebertscore: Evaluating code generation with pretrained models of code. *arXiv preprint arXiv:2302.05527* (2023).
- [124] Xin Zhou, Sicong Cao, Xiaobing Sun, and David Lo. 2025. Large Language Model for Vulnerability Detection and Repair: Literature Review and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (2025), 145:1–145:31. doi:10.1145/3708522
- [125] Xin Zhou, Kisub Kim, Ting Zhang, Martin Weyssow, Luis F Gomes, Guang Yang, and David Lo. 2025. An LLM-as-Judge Metric for Bridging the Gap with Human Evaluation in SE Tasks. *arXiv preprint arXiv:2505.20854* (2025).
- [126] Xin Zhou, Ting Zhang, and David Lo. 2024. Large Language Model for Vulnerability Detection: Emerging Results and Future Directions. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results, NIER@ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 47–51. doi:10.1145/3639476.3639762
- [127] Yilun Zhou, Austin Xu, Peifeng Wang, Caiming Xiong, and Shafiq Joty. 2025. Evaluating judges as evaluators: The jets benchmark of llm-as-judges as test-time scaling evaluators. *arXiv preprint arXiv:2504.15253* (2025).
- [128] Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, et al. 2024. Agent-as-a-judge: Evaluate agents with agents. *arXiv preprint arXiv:2410.10934* (2024).
- [129] Terry Yue Zhuo. 2023. ICE-Score: Instructing Large Language Models to Evaluate Code. *arXiv preprint arXiv:2304.14317* (2023).
- [130] Terry Yue Zhuo, Qian Liu, Zijian Wang, Wasil Uddin Ahmad, Binyuan Hui, and Loubna Ben Allal. 2025. NLP+Code: Code Intelligence in Language Models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Tutorial Abstracts*, Valentina Pyatkin and Andreas Vlachos (Eds.). Association for Computational Linguistics, Suzhou, China, 9–11. doi:10.18653/v1/2025.emnlp-tutorials.4
- [131] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widayarsi, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. 2024. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877* (2024).

Received 30 September 2025; revised 11 January 2026; accepted 29 January 2026